Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Projekt zespołowy – laboratorium 2

Implementacja bazy danych szeregów czasowych w języku Rust

Architektura systemu i szczegółowy projekt techniczny

Autorzy: Hubert Błonowski
Jan Zaborski

Promotor: mgr inż. Paweł Sobótka

Wersja dokumentu: 1.3
Data: 2 listopada 2026 r.

`https://github.com/janzaborski/rust-tsdb`

Streszczenie

Projekt jednowęzłowej TSDB w Rust obejmuje moduły, model danych, HTTP, podzbiór PromQL i Grafanę. Jedna TSMap z osobnymi blokadami TS pozwala współbieżnie zapisywać, a rotacja bloków zachowuje dostępność podczas flush. Buforowany WAL, flusher i syncer realizują tryby None, Written i Synced. Kontrakty, formaty, indeksy i recovery umożliwiają niezależną implementację magazynu, zapytań i HTTP. To **specyfikacja docelowa**, nie opis prototypu ani wyniki pomiarów. Punktem wyjścia jest lab1; doprecyzowania wymagań wskazano jawnie.

Historia zmian

Wersja	Data	Autorzy	Zakres zmiany
1.0	24.10.2026	H. Błonowski, J. Zaborski	Architektura, API, formaty, diagramy, GUI i plan weryfikacji.
1.1	24.10.2026	H. Błonowski, J. Zaborski	TSMap/TS, współbieżność bez shardów, rotacja bloków i flush w tle.
1.2	24.10.2026	H. Błonowski, J. Zaborski	Buforowany WAL, bajtowy LSN, poziomy trwałości, checkpoint i recovery.
1.3	02.11.2026	H. Błonowski, J. Zaborski	Krótszy opis; UML przypadków użycia, klas, wdrożenia i stanów.

Spis treści

Streszczenie . . . i
Historia zmian . . . i
1 Cel, zakres i decyzje architektoniczne . . . 1
2 Architektura systemu . . . 2
3 Model danych i kontrakty modułów . . . 5
4 Zapis, buforowanie i obsługa opóźnionych danych . . . 7
5 Organizacja danych na dysku . . . 9
6 Checkpoint i odzyskiwanie po awarii . . . 14
7 Język i wykonywanie zapytań . . . 17
8 Komunikacja i publiczne API . . . 19
9 Technologie, konfiguracja i środowisko . . . 22
10 Wizja interfejsu użytkownika i demonstracja . . . 25
11 Weryfikacja architektury i kryteria odbioru . . . 27
A Słownik . . . 30
B Budowanie dokumentu . . . 31
Źródła . . . 31

1 Cel, zakres i decyzje architektoniczne

1.1 Cel, odbiorcy i granice systemu

Baza przyjmuje pomiary aplikacji, przechowuje je we własnym silniku i udostępnia historię oraz agregacje w Grafanie. Projekt dla autorów, promotora i recenzenta określa zachowanie docelowe; przykłady i cele wydajnościowe nie są wynikami prototypu.

Zakres: jednowęzłowy silnik Rust, HTTP, dokładne dopasowanie etykiet, pięć agregacji czasowych i demonstracja w Compose. Grafana i generatory są klientami zewnętrznymi. Poza zakresem pozostają klaster, replikacja, kompresja, retencja, kompaktowanie, alertowanie, pobieranie `/metrics` i własny frontend. Dane rosną do usunięcia środowiska; kopia zapasowa wymaga zatrzymania bazy i skopiowania całego katalogu.

1.2 Doprecyzowania względem laboratorium 1

Wymaganie	Przyjęta interpretacja i konsekwencja
FR-01	Atomowa paczka: błędny punkt odrzuca całość; utrata odpowiedzi po zapisie nie wycofuje danych.
FR-03	Widoczność head poprzedza odpowiedź. Domyślne Written oznacza zapis ramki do systemu operacyjnego, Synced także synchronizację, a jawne None dopuszcza utratę po awarii procesu.
FR-04	Zakres to obliczenia na siatce czasu. Selektor wybiera ostatnią próbkę; agregacja wymaga funkcji.
FR-08	Czas buforowania biegnie od przyjęcia najstarszej próbki generacji. Flush całej generacji może wcześniej usunąć młodsze dane z RAM; limit pamięci może go przyspieszyć.
FR-09	Out-of-order wyłącznie w otwartym buforze: obowiązują jednocześnie okno opóźnienia i granica zamknięcia.
NFR-03	RSS $\leq$ 1 GiB (nie Gb); 288 MB surowych próbek nie obejmuje zapasowej pojemności, indeksów ani zapytań.
NFR-04	Brak utraty przy awarii procesu dotyczy Written/Synced, przy sprawnym OS i nośniku. None mierzy się osobno; błąd I/O WAL kończy proces. Fizyczne uszkodzenie nośnika jest poza gwarancją.

1.3 Rejestr głównych decyzji

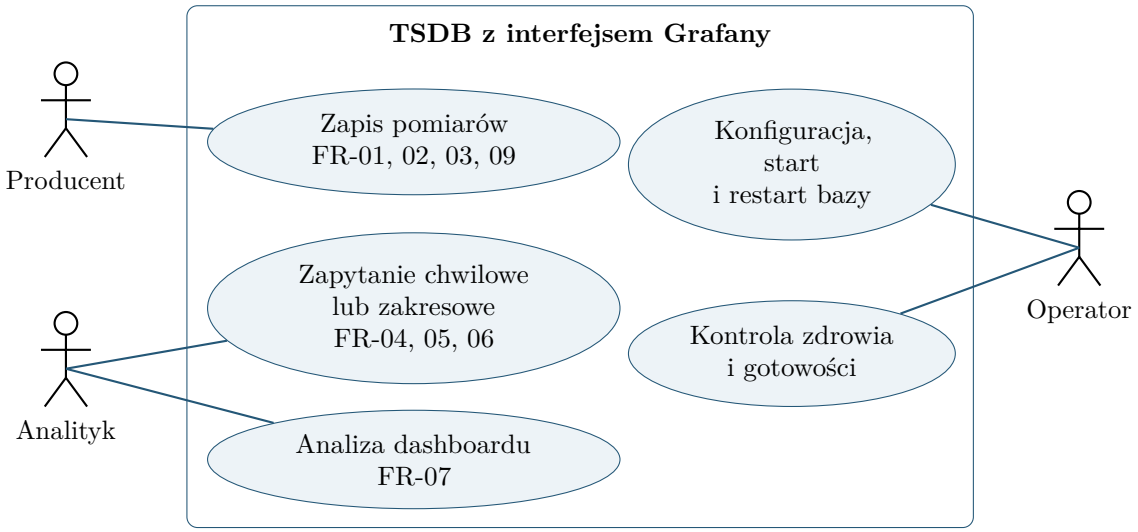
ID	Decyzja, uzasadnienie i koszt
DA-01	Modularny monolit Rust: wywołania wewnątrz procesu upraszczają transakcje i diagnostykę; skalowanie do jednego węzła.
DA-02	Jedna TSMap, osobna blokada każdego TS: równoległe zapisy rozłącznych szeregów, krótkie blokady katalogu, bez shardów.
DA-03	Posortowane, niekompresowane bloki aktywne i zamrożone: out-of-order i dostępność odczytu podczas flush.
DA-04	Wspólne generacje head: krótka bariera wyznacza checkpoint; jeden flush w tle pozwala dalej zapisywać.

ID	Decyzja, uzasadnienie i koszt
DA-05	Grafana jako jedyne GUI: standardowe źródło Prometheus, zgodność sprawdzana z przypiętą wersją.
DA-06	Zamknięte zakresy są niezmienne: proste indeksy i rozłączne bloki szeregu; wcześniejszy flush może spowodować odrzucenie punktu.
DA-07	Jeden buforowany WAL, flusher i syncer: rotowane pliki; oczekiwanie na I/O poza blokadami TS.
DA-08	Atomowa paczka i spójny widok wybranych TS: blokady rosnąco po ID; duża operacja może ograniczyć równoległość.
DA-09	Written domyślnie, None/Synced per żądanie: widoczność i trwałość są odrębnymi właściwościami.

2 Architektura systemu

2.1 Przypadki użycia i aktorzy

Producent wysyła pojedyncze pomiary lub atomowe paczki. Analityk wybiera czas i serwis w Grafanie albo odpytuje API. Operator uruchamia, konfiguruje i restartuje bazę oraz kontroluje jej stan.

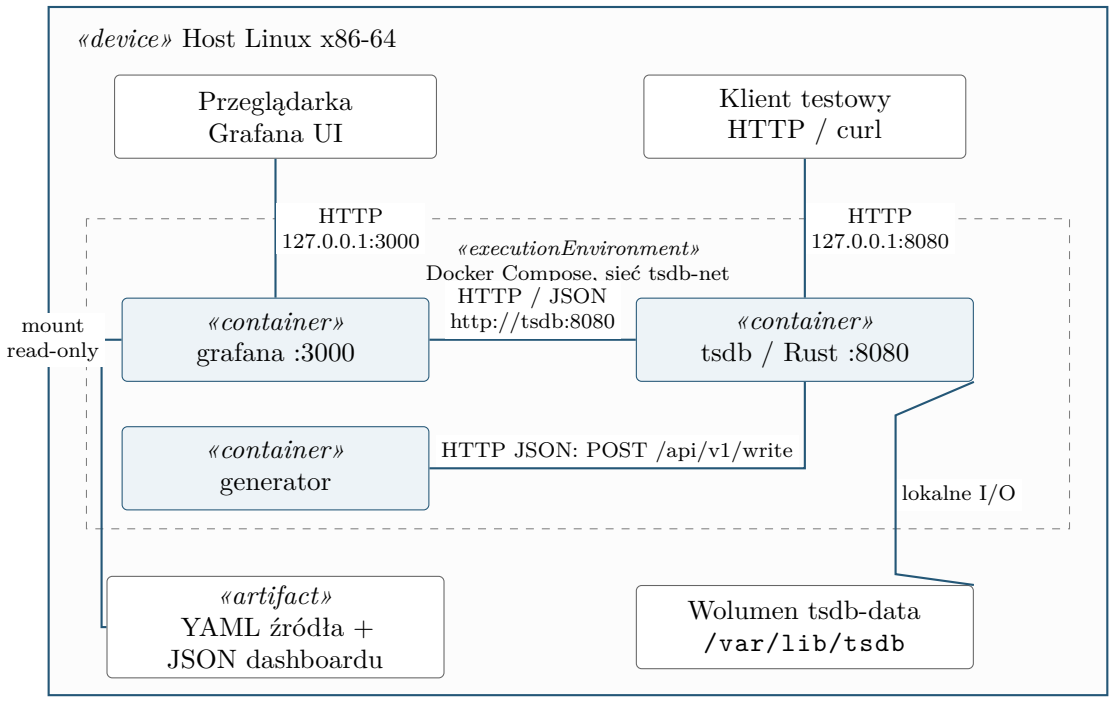


Rysunek 1: Diagram przypadków użycia UML. Flush (FR-08) jest automatycznym mechanizmem silnika, a nie osobnym działaniem użytkownika.

2.2 Wdrożenie i granice procesów

System działa na jednym komputerze. Przeglądarka łączy się z Grafaną; serwer Grafany odpytuje TSDB, a generator wysyła HTTP JSON. W środowisku nie ma serwera Prometheus.

Jeden proces ma wyłączny dostęp do katalogu: systemowa blokada pliku `LOCK` trwa tak długo jak jego otwarty deskryptor; pozostały po awarii plik nie oznacza aktywnej blokady. Sieciowe systemy plików nie są platformą docelową.

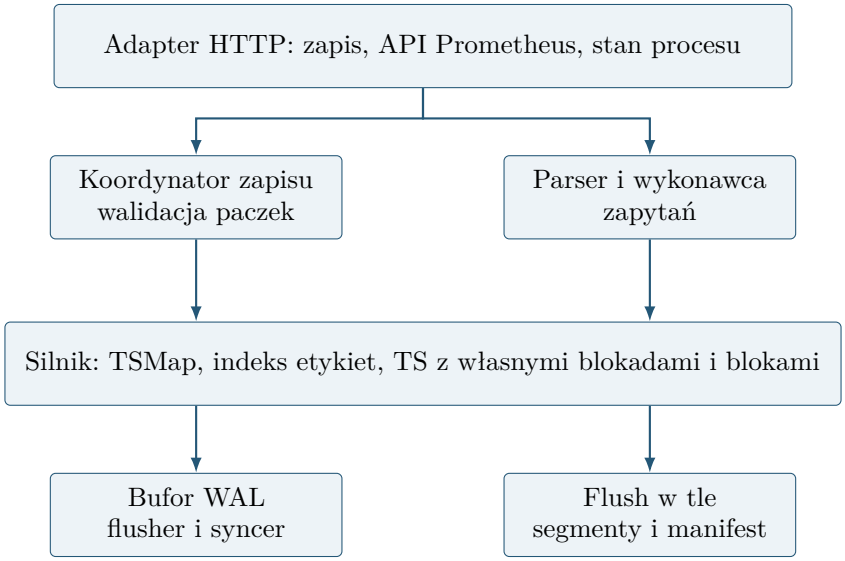


Rysunek 2: Diagram wdrożenia UML z zasobami lokalnymi. Porty hosta są publikowane wyłącznie na loopback; kontenery używają DNS tsdb. Szczegóły dostępu określa sekcja 9.3.

2.3 Warstwy, moduły i zależności

Warstwa HTTP odpowiada za protokół, warstwa aplikacyjna za operacje zapisu i zapytań, a silnik za dane i pliki. Parser zależy wyłącznie od typów języka. Silnik nie zna kodów HTTP, Grafany ani składni PromQL.

Moduł	Odpowiedzialność, interfejs i zależności
HTTP	Dekodowanie, limity, błędy i odpowiedzi; Axum/Serde, wywołania zapisu i zapytań.
Model i walidacja	Wspólne typy, kanonizacja klucza, limity nazw/etykiet i wartości; bez HTTP i plików.
Koordynator zapisu	Pula, kolejność blokad, walidacja/rezerwacja; TSMap, TS, WAL i bariera head.
TSMap i indeks	Klucz ↔ TS, rejestracja i indeks etykiet; metadane pod krótką blokadą katalogu.
TS i bloki	Posortowane próbki, duplikaty, granice czasu, aktywny/zamrożony blok; blokada per TS, globalny budżet RAM.
WAL	Bufor, bajtowy LSN, Written/Synced i rotacja; mutex, flusher/syncer, watch, bez blokad TS w I/O.
Magazyn segmentów	Lokalne pliki, CRC, indeksy, manifest/checkpoint; koordynator flush.
Parser	Tekst → AST/błąd z pozycją; typy języka, bez dostępu do danych.
Wykonawca zapytań	Dobór TS, plan, agregacje i budżety; AST i interfejs odczytu, bez dostępu do WAL.
Uruchomienie i diagnostyka	Konfiguracja, blokada katalogu, recovery, gotowość, logi i zakończenie; zarządza cyklem życia modułów.



Rysunek 3: Zależności logiczne. Konfiguracja i diagnostyka są współdzielonymi usługami pomocniczymi.

2.4 Współbieżność i spójność

Tokio obsługuje sieć; ograniczona pula wykonuje pamięciową część zapisu (domyślnie cztery zadania). Osobne `RwLock` TS pozwalają równolegle zmieniać rozłączne szeregi. Wszystkie operacje przestrzegają poniższej kolejności; nie utrzymują blokady katalogu podczas pobierania blokad TS.

Operacja	Blokady i punkt publikacji
Zapis	Dla nowych TS: mutex rejestracji; następnie współdzielona bariera generacji i wyłączne blokady TS rosnąco po ID. Walidacja i rezerwacja zasobów → kodowanie → enqueue WAL → publikacja całej paczki przed zwolnieniem którejkolwiek blokady TS.
Enqueue	Mutex bufora obejmuje wyłącznie kopię kompletnej ramki i przydział LSN. Może wystąpić oczekiwanie na mutex lub budżet; brak I/O, aktualizacji head i oczekiwania na trwałość pod tym mutexem. Mechanizm nie jest lock-free.
Trwałość	Po publikacji zwolnić blokady, barierę i miejsce w puli. HTTP czeka asynchronicznie przez watch; None odpowiada od razu. Flusher i syncer nie pobierają blokad TS ani bariery head (sekcja 5.2).
ReadView	Z TSMap pobrać uchwyty i wersję katalogu, zwolnić katalog, pobrać współdzielone blokady wszystkich wybranych TS rosnąco po ID (`try_read` z timeoutem). Zmiana atomowej wersji katalogu wymaga zwolnienia blokad i ponowienia wyboru. Skopiować potrzebne aktywne próbki i uchwyty niezmiennych bloków/segmentów; zwolnić blokady.
Zamrożenie	Wyłączna bariera czeka na części enqueue–head append, ustala C i zamraża bloki. Zwalnia barierę przed I/O. Nowe aktywne bloki przyjmują zapis podczas flush (sekcja 6); rotacja WAL jest niezależna.

ReadView odpowiada jednej chwili stanu wybranych szeregów, bez części paczki; całe query_range używa tego samego widoku. Może zobaczyć dane klienta oczekującego na trwałość. Nie ma jednego globalnego LSN widoku: publikacje rozłącznych TS mogą kończyć się poza kolejnością WAL, a publikacje wspólnego TS zachowują tę kolejność. Odczyt po odpowiedzi sukcesu widzi zapis w swoim zakresie.

3 Model danych i kontrakty modułów

3.1 Tożsamość szeregu i reprezentacja punktu

Klucz szeregu składa się z nazwy metryki oraz uporządkowanej listy par etykiet. Klucze etykiet są unikalne; sortowanie jest leksykograficzne według bajtów UTF-8. Nie normalizuje się wielkości liter ani Unicode. Porównanie pełnego klucza rozstrzyga tożsamość również przy kolizji funkcji skrótu.

Typ / pole	Reprezentacja	Reguła
`SeriesKey`	Metryka i posortowane pary napisów.	Nazwa metryki: `[a-zA-Z_:][a-zA-Z0-9_:]*`, do 128 bajtów.
Etykieta	Nazwa i wartość UTF-8.	Nazwa: `[a-zA-Z_][a-zA-Z0-9_]*`, do 64 bajtów; wartość do 256 bajtów, maksymalnie 16 etykiet.
Nazwy zastrzeżone	Prefiks `__`.	Nie są dopuszczane w etykietach zapisu. `__name__` jest wirtualną etykietą nazwy metryki w odpowiedziach.
`SeriesId`	`u64`, od 1.	Przydzielany rosnąco przy rejestracji w TSMap; nie jest hashem ani częścią publicznego API.
`Timestamp`	`i64`, milisekundy od epoki Unix.	Zakres przyjmowania: 0 do $2^{53} - 1$ ms, aby zachować dokładność całkowitą w klientach JSON.
`Sample`	`timestamp:i64`, `value:f64`.	16 bajtów w bloku; wartość wejściowa musi być skończona. Normalizacja -0 do $+0$.
`Lsn`	`u64`, od 1.	Pozycja końca rekordu w logicznym strumieniu bajtów WAL, bez nagłówków plików. Rośnie o długość ramki; 0 oznacza pusty strumień.

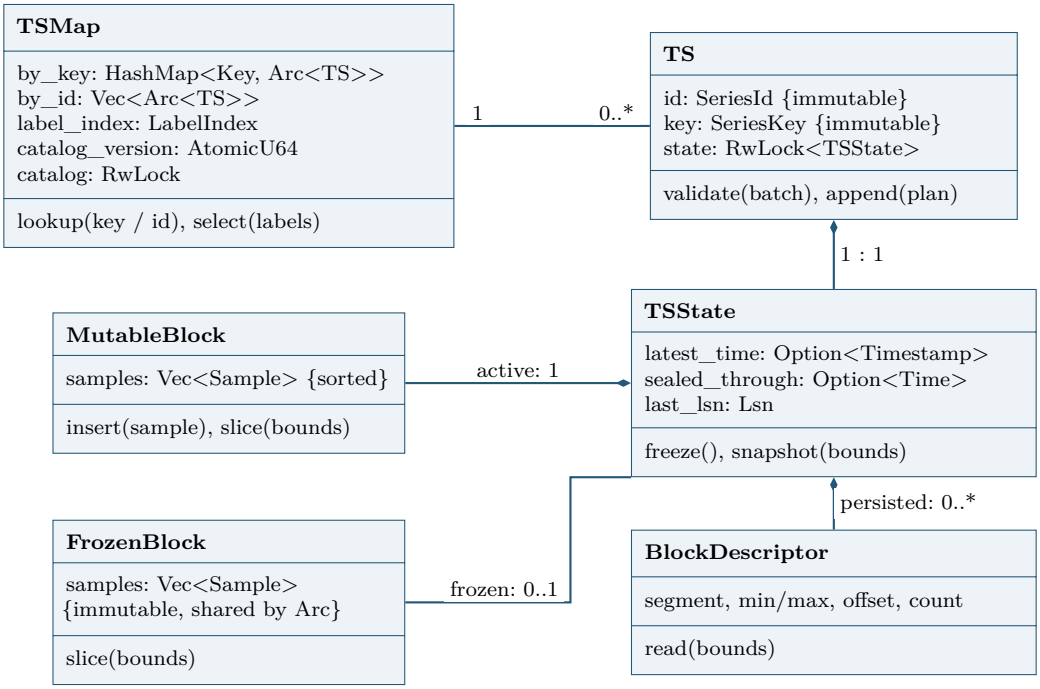
Pusta wartość etykiety jest kanonizowana przez usunięcie tej etykiety. Dzięki temu porównanie z pustym napisem może objąć również szeregi bez danej etykiety. Powtórzona nazwa klucza w obiekcie JSON jest błędem przed kanonizacją, także gdy parser JSON domyślnie potrafiłby ją nadpisać. Nieznane pola wejściowe są odrzucane. Wartości metryk i etykiet w przykładach stosują konwencje Prometheusa; projekt celowo ogranicza dozwolone nazwy do wskazanej składni ASCII.

3.2 Struktury TSMap, TS i bloki

Inspiracją jest oddzielenie katalogu, synchronizacji szeregu i bloków otwartych/zamkniętych w Gorilla, sekcje 4.2–4.3 [2]. Tutaj jedna TSMap obsługuje całą bazę, a bloki bez kompresji zapewniają dostępność podczas flush. Buforowany WAL ma jawne poziomy trwałości.

Jeden RwLock katalogu chroni mapę, wektor i indeks etykiet. `by_id` to uporządkowana lista do skanowania przy zamrożeniu i binarnego lookup; luki ID są dopuszczalne, więc ID nie jest indeksem wektora. Szeregów nie usuwa się, a ID nie wykorzystuje ponownie.

Rejestracja nowych TS wymaga mutexu i ponownego lookup. Przygotowuje je poza publiczną mapą; ich definicje i pierwsze próbki trafiają do jednej ramki WAL. Dopiero po enqueue i inicjalizacji head krótka wyłączna blokada katalogu publikuje TS oraz indeksy, zwiększając wersję przed zwolnieniem blokad TS. Katalog nie jest zablokowany podczas enqueue/I/O. Konkurent może użyć TS dopiero po publikacji, więc nie zaloguje próbek przed definicją. Istniejące TS nie wymagają mutexu rejestracji; nieudana rezerwacja może pozostawić lukę ID, nigdy pusty publiczny TS.



Rysunek 4: Diagram klas modelu head (typy Rust, bez dziedziczenia). Romb oznacza własność; zwykłe linie także współdzielone uchwyty. Key = SeriesKey, Time = Timestamp; Sample = timestamp: i64 + value: f64 (16 B).

3.3 Duplikaty i atomowość paczek

W szeregu jest najwyżej jedna wartość dla czasu. Identyczna próbka w otwartym buforze to duplikat bez zmian; inna wartość odrzuca całą paczkę jako konflikt, również przy powtórzeniach wewnątrz paczki. Zamknięty zakres odrzuca nawet identyczny duplikat bez odczytu dysku: API nie daje bezterminowej idempotencji ani rejestru jej kluczy. Po utracie odpowiedzi wynik można sprawdzić zapytaniem.

Walidacja tworzy plan całej paczki; definicje i próbki mają wspólną trwałość ramki. Paczka samych duplikatów nie tworzy rekordu, lecz czeka na wybrany poziom dla maksimum `last_lsn` dotkniętych TS. Ponowienie None jako Synced wymaga więc rzeczywistego sync. LSN odpowiedzi nie obiecuje, że mniejsze LSN są już widoczne w rozłącznych TS.

3.4 Kontrakty wewnętrzne

Notacja nie narzuca osobnych crate'ów. Operacje zwracają `Result<T,Error>` z typowanym błędem właściwego modułu. Selektory, AST i widoki są pożyczane; `QueryBudget` jest mutowalny.

Operacja → wynik	Kontrakt
`canonicalize(Measurement)` → `CanonicalPoint`; `parse(&str)` → `Expr`	Kanonizacja pomiaru; AST selektora, funkcji z oknem lub skalara.
`submit(Vec<CanonicalPoint>,Durability)` → `WriteReceipt`	Sukces: widoczna cała paczka i osiągnięta trwałość. Receipt: inserted, duplicates, lsn, durability.
`snapshot(Selector,TimeBounds,QueryBudget)` → `ReadView`	Wersja katalogu, kopie aktywnych próbek i uchwyty bloków/segmentów; reguły sekcji 2.4.

Operacja → wynik	Kontrakt
`read_series(ReadView,SeriesId,TimeBounds)` → `SampleCursor`	Próbki chronologicznie, źródło ukryte.
`evaluate(Expr,EvalGrid,ReadView,QueryBudget)` → `QueryResult`	Budżet czasu, pamięci, próbek i wyniku.
`wal.enqueue(EncodedRecord)` → `Lsn`; `wal.wait_for(Lsn,Durability)` → `()`	Enqueue bez I/O, LSN końca ramki; wait po publikacji i zwolnieniu blokad.
`freeze_head()` → `FrozenGeneration`; `persist(FrozenGeneration)` → `Checkpoint`	Generacja zawiera numer head, cutoff LSN, katalog z chwili zamrożenia i uchwyty bloków; persist działa w tle.
`recover(Path)` → `RecoveredState`	Stan po walidacji checkpointu i replay WAL.

Błędy kontraktów: canonicalize/parse/submit zwracają odpowiednio `ValidationError`/`ParseError`/`WriteError`; snapshot/read – `ReadError`, evaluate – `QueryError`, enqueue/wait – `AdmissionError`/`WaitError`, freeze/persist – `StorageError`, recover – `RecoveryError`.

Walidacja, konflikt i brak budżetu kończą się przed enqueue. Po nim publikacja używa zarezerwowanych zasobów i nie ma zwykłego rollbacku; timeout/anulowanie nie wycofują danych. I/O WAL kończy proces; pozostałe błędy mapuje sekcja 8.5.

3.5 Podział odpowiedzialności zespołu

Osoba	Obszar odpowiedzialności
Jan Zaborski	WAL, segmenty, manifest, indeksy dyskowe, checkpoint i recovery.
Hubert Błonowski	Parser, wykonawca zapytań, adapter HTTP, integracja i dashboard Grafany.
Obaj autorzy	Model danych, bufor i granice jego interfejsu, przegląd kontraktów, testy integracyjne, pomiary i dokumentacja swoich podsystemów.

Wykonawcę zapytań można rozwijać z pamięciową implementacją interfejsu odczytu. Magazyn można testować bez HTTP i parsera. Adapter HTTP korzysta z ustalonych typów wyników i błędów. Wspólna akceptacja zmian modelu i kontraktów poprzedza ich wdrażanie po obu stronach.

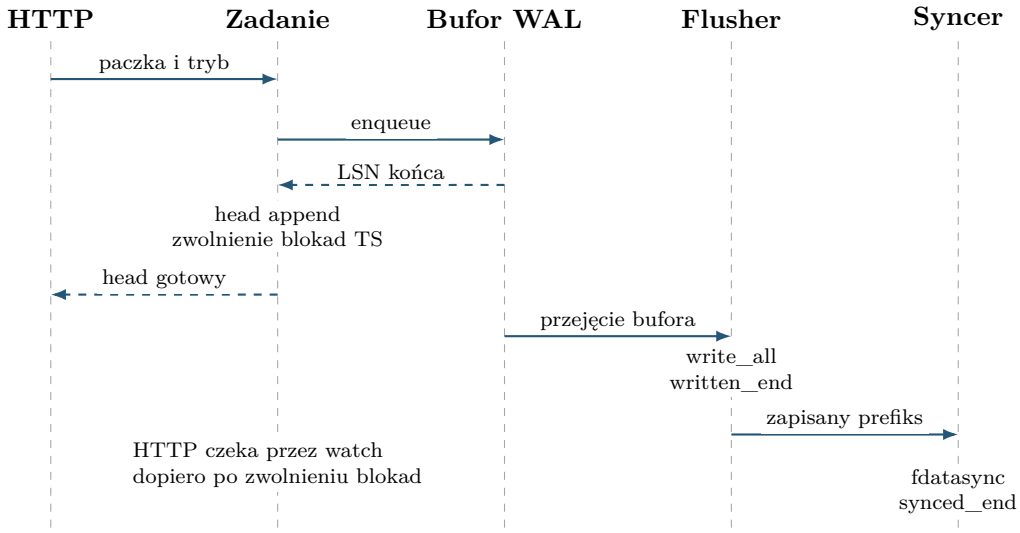
4 Zapis, buforowanie i obsługa opóźnionych danych

4.1 Algorytm przyjęcia paczki

1. Przed wczytaniem ciała zarezerwować miejsce przyjęcia; sprawdzić rozmiar/tryb, zdekodować JSON i kanonizować punkty.
2. Pobrać uchwyty i blokady według sekcji 2.4. Zweryfikować całą paczkę (duplikaty, opóźnienie, zamknięcie) i zarezerwować head, indeksy, scratch oraz WAL.
3. Brak pamięci/konieczność flush: zwolnić wszystkie blokady, czekać i ponowić pełną walidację. Paczka niemieszcząca się w pustym budżecie: 422; przejściowy brak po maks. 1 s: 503. Oba błędy poprzedzają enqueue.
4. W thread-local scratch zakodować definicje, potem nowe próbki. Pod mutexem WAL skopiować pełną ramkę i uzyskać LSN.
5. Pod blokadami TS opublikować całą paczkę i nowe indeksy; zmienionym TS ustawić `last_lsn`.

6. Zwolnić blokady, barierę i worker. None odpowiada; Written/Synced asynchronicznie czekają na odpowiedni prefiks.

Recovery może odtworzyć kompletną ramkę także bez odpowiedzi klientowi lub bez zakończonego head append. Atomowość publikacji nie łączy jej w jedną transakcję z fizyczną synchronizacją.



Rysunek 5: Część pamięciowa żądania i niezależne zadania I/O. None kończy się po publikacji head, Written po write, Synced po sync.

4.2 Warunki przyjmowania out-of-order

H_s to największy przyjęty czas szeregu (także na dysku), S_s – największy czas danych zamrożonych/opublikowanych, a W – okno opóźnienia. Dla każdego nowego punktu:

$$t > S_s \quad \wedge \quad t \geq H_s - W.$$

S_s pomija się przed pierwszym zamrożeniem, H_s dla nowego TS. Całą paczkę waliduje się względem H_s sprzed paczki, niezależnie od kolejności JSON; po publikacji H_s jest maksimum starego i nowych czasów. Arytmetyka jest sprawdzana, ujemna dolna granica oznacza zero.

Kolejność decyzji: zamknięty zakres → duplikat/konflikt w otwartym buforze → test W dla nowego punktu. Identyczny otwarty duplikat nie staje się nowym zapisem. Próbka może wyprzedzać zegar serwera najwyżej o 60 s, co chroni H_s przed błędnym przesunięciem. Zegar ścienny służy temu testowi i domyślnemu czasowi zapytania; pobyt w RAM i timeouty używają zegara monotonicznego.

Stan	Czas punktu	Wynik
$H = 12{:}10$, $S = 12{:}00$, $W = 5$ min	12:06 / 12:04	Przyjęcie / przekroczone W.
Po zamrożeniu $S = 12{:}10$	12:09 / 12:11	Zakres zamknięty / nowy aktywny blok (także podczas flush).

Przy $W = 0$ paczka może zawierać kilka czasów, ale żaden nowy nie może poprzedzać starego H_s; punkty paczki nie ustalają kolejności przyjęcia. Profil standardowy: $W = 5$ min.

4.3 Organizacja i limit pamięci

Posortowany wektor dopisuje rosnące czasy na końcu; out-of-order wymaga wyszukiwania binarnego i liniowego przesunięcia. Dwa wyszukiwania wyznaczają wycinek odczytu. Pojemność rośnie przez

`try_reserve_exact` w wielokrotnościach 4096 próbek; rozlicza się faktyczną pojemność, także zapas i przejściowe współistnienie starej/nowej alokacji.

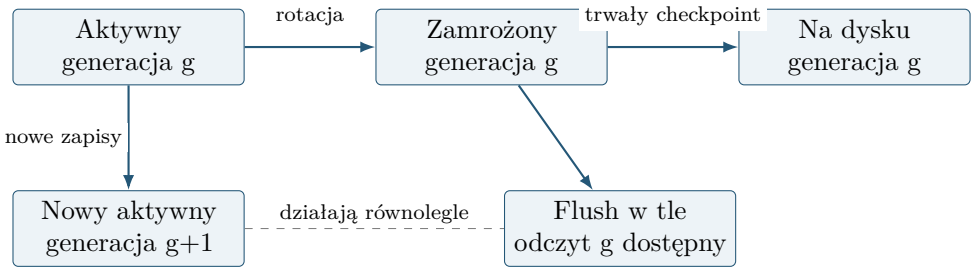
Budżet head obejmuje aktywne, zamrożone i podtrzymywane przez ReadView bloki, każdy współdzielony blok raz do ostatniego uchwytu. Katalog, przyjęcia, WAL i zapytania mają osobne budżety. Rezerwacja poprzedza enqueue; błąd nie publikuje próbek. Za mały limit minimalnej alokacji oznacza błąd konfiguracji/paczki, bez nieskończonego flush.

Zamrożenie przenosi wektor do `Arc<FrozenBlock>` bez pełnej kopii. Nowy aktywny blok alokuje w miarę napływu danych. Zapytania kopiują wyłącznie potrzebne aktywne próbki; niezmienne dane współdzielą. Po publikacji segmentu starszy widok może nadal podtrzymywać RAM, co uwzględnia się w budżecie i pomiarach RSS.

4.4 Wyzwalanie i przebieg zrzutu

Timer co 1 s zleca flush po upływie czasu pobytu najstarszej próbki generacji w RAM; termin dotyczy rozpoczęcia, nie zwolnienia pamięci. Inne wyzwalacze: przekroczenie 80% budżetu lub brak miejsca na paczkę. Pustej generacji nie zrzuca się.

Koordynator zamraża razem niepuste bloki, bez rotacji WAL. Po zwolnieniu bariery nowe zapisy działają równolegle z jednym flush w tle (sekcja 6). Do jego zakończenia następny flush pozostaje oczekujący, bez kolejki zamrożonych generacji. Przy twardym limicie paczki krótko czekają bez blokad lub dostają 503; dostępność podczas flush podlega ograniczonej pamięci i szybkości dysku.



Rysunek 6: Rotacja tworzy nowy aktywny blok przed zapisem poprzedniego na dysk. Kolejna rotacja czeka na zakończenie poprzedniego flush.

5 Organizacja danych na dysku

5.1 Katalog i wspólne reguły formatu

```
data/LOCK
data/MANIFEST
data/wal/wal-00000000000000000001.log
data/segments/segment-00000000000000000001.bin
```

Obie numeracje mają po 20 cyfr: segmenty to generacje head, WAL to niezależna kolejność plików. Nazwy tymczasowe kończą się `.tmp`. Jeden WAL jest aktywny, starsze czekają na checkpoint. Manifest wskazuje segmenty danych; replay obejmuje kotwicę i wszystkie późniejsze opublikowane WAL, także utworzone podczas flush.

Formaty: little-endian, IEEE 754 f64, bez paddingu/układu struktur Rust; napis = `u32byte_len` + UTF-8. Długości, offsety i przepełnienia sprawdza się przed alokacją/odczytem. Segment ma wersję 2, WAL/manifest 3 (bajtowy LSN, niezależne numeracje). Nieznana wersja, niezerowe flagi, błędne długości/UTF-8 zatrzymują start; migracja i zgodność z plikami prototypu nie są zapewnione.

CRC-32/IEEE [10], zapis u32 LE, ma wektor `123456789->0xCBF43926`; wykrywa uszkodzenia, nie uwierzytelnia.

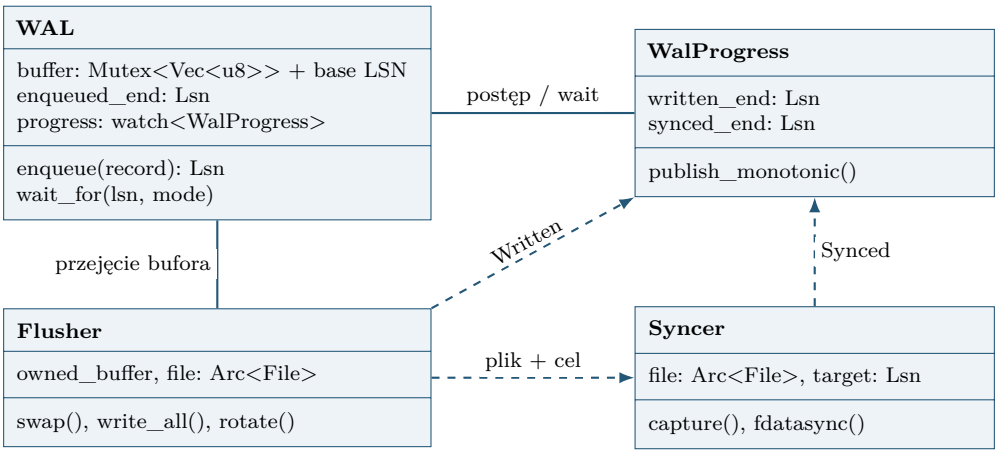
5.2 Bufor WAL, flusher i syncer

Jeden logiczny strumień utrzymuje monotoniczne końce ciągłych prefiksów:

$$0 \leq synced_end \leq written_end \leq enqueued_end.$$

Cel żądania to `lsn=record_end`. Prefiksy opisują WAL, nie stan publikacji head; do checkpointu potrzebna jest osobna bariera.

Tryb	Warunek sukcesu po publikacji head i gwarancja
None	Bez oczekiwania na I/O: awaria procesu może utracić paczkę; zachowane pełne ramki mogą wrócić w replay.
Written	`written_end>=lsn`: cała ramka w OS/page cache. Przetrwa awarię samego procesu przy sprawnym hoście i nośniku.
Synced	`synced_end>=lsn`: zakończone fdatasync i trwała publikacja nazwy pliku; gwarancje lokalnego systemu plików.



Rysunek 7: Diagram klas współpracy WAL. Flusher i Syncer to niezależne wątki; przerywane strzałki oznaczają zależność, linie ciągłe asocjację. Durability = None | Written | Synced.

Enqueue.
Żądanie koduje w thread-local scratch; pod mutexem kopiuje kompletną ramkę do bufora i zwiększa enqueued_end. Nie wykonuje I/O (kolejność publikacji: sekcja 4).

Flusher.
Jedyny dopisujący do plików. Bez waiterów: zapis co najwyżej co 10 ms lub po 1 MiB. Written/Synced rejestrują cel i budzą go wcześniej; od pobudzenia może zebrać grupę przez 200 μs (parametr pomiarowy, nie cel opóźnienia HTTP). Pod mutexem podmienia bufor na pusty i zachowuje zakres LSN; poza nim sekwencyjnie wykonuje `write_all`. Pojedynczy write może być krótki [12]. Written publikuje wyłącznie dla pełnych ramek, także przed rotacją; nowe żądania już dopisują do nowego bufora.

Syncer.
Co 1 s lub przy niezaspokojonym celu Synced: jeśli cel jest w RAM, budzi flusher i czeka na Written. *Przed* fdatasync zapamiętuje uchwyt pliku i zapisany cel; po sukcesie potwierdza tylko ten prefiks, bez późniejszych dopisań.

Oczekiwanie.
Wspólny watch przechowuje ostatni stan obu granic [15]; aktualizacje zachowują monotoniczność. Waiter sprawdza stan przed czekaniem i po zmianie, nie wymaga każdego zdarzenia. Zamknięcie kanału nie daje sukcesu. Timeout 30 s/anulowanie kończą wait bez rollbacku.

Rotacja.
Przed ramką przekraczającą 128 MiB razem z nagłówkiem: nie dzielić rekordu; zsynchronizować stary plik, opublikować Synced; utworzyć nowy jako .tmp, zapisać/synchronizować nagłówek, rename i sync katalogu wal/, dopiero potem pisać ramki. Sam sync pliku nie utrwala jego nazwy [13]. Krótka blokada stanu I/O uzgadnia plik i cel, nigdy nie obejmuje fdatasync. Uchwyt pozostaje ważny; Synced nowego pliku wymaga trwałego prefiksu poprzednich. Rotacja nie pobiera bariery head ani blokad TS.

Pamięć i błędy.
32 MiB rozlicza rzeczywiste pojemności bufora aktywnego i przejętego; scratch podlega budżetowi przyjęcia, także między żądaniami. Brak miejsca: oczekiwanie bez blokad TS lub 503 przed enqueue, bez nieograniczonej kolejki. Każdy błąd write, fdatasync lub publikacji WAL: raport i natychmiastowe zakończenie procesu (fail-stop), bez sukcesu nieosiągniętym celom. None mogło już odpowiedzieć. Replay odtwarza poprawny prefiks; uszkodzony nośnik/pełna ramka CRC mogą zatrzymać start.

`sync_file_range` pozostaje opcjonalną, osobno mierzoną optymalizacją writeback; nie zastępuje fdatasync [14]. Okres 1 s nie ogranicza straty None: zależy ona też od opóźnienia flushera i danych pozostających w RAM.

5.3 Format dziennika i bajtowy LSN

Plik ma następujący nagłówek:

Offset	Rozmiar	Pole	Znaczenie
0	8 B	magic	ASCII `RSTWAL03`.
8 / 10 / 12	2 / 2 / 4 B	version / flags / header_len	`u16=3` / `u16=0` / `u32=32`.
16	8 B	segment_id	`u64`; numer pliku zgodny z nazwą.
24	8 B	base_lsn	Logiczna pozycja początku pierwszej ramki.
32	zmienny	frames	Pełne ramki w kolejności enqueue.

LSN to liczba bajtów ramek bez nagłówków: pozycja pierwszego bajtu *po* rekordzie. Dla fizycznego końca p:

$$record_end = base_lsn + (p - 32).$$

Pusta baza ma base=0; następny plik zaczyna się na końcu poprzednika, bez dodawania 1. Przyrost = długość ramki; arytmetyka sprawdzana.

Ramka ma układ `len|crc32|kind|flags|reserved|payload`:

Offset	Rozmiar	Pole	Znaczenie
0	4 B	record_len	Cała długość ramki, od 20 B do 8 MiB.
4	4 B	crc32	CRC bajtów 0–3 i 8–końca ramki, z pominięciem pola CRC.
8 / 9 / 10	1 / 1 / 2 B	kind / flags / reserved	`u8=1` (paczka) / `u8=0` / `u16=0`.

Offset	Rozmiar	Pole	Znaczenie
12	record_len −12	payload	Definicje szeregów, a następnie próbki.

Payload w kolejności:

1. `new_series_count:u32`, potem definicje rosnąco po ID: `id:u64`, napis metryki, `label_count:u16`, kanonicznie posortowane pary napisów nazwa–wartość.
2. Dodatni `point_count:u32`, potem próbki rosnąco po ID/czasie: `series_id:u64,timestamp:i64,value_bits:u64` (24 B).

Duplikatów bez zmian nie zapisuje się; każda definicja ma próbkę w tej ramce. ID musi pochodzić z checkpointu, wcześniejszej ramki lub jej definicji; brak jest błędem, bez odkładania/pomijania.

Próbka istniejącego TS: payload $4 + 4 + 24 = 32$ B, ramka $12 + 32 = 44$ B; jako pierwsza w pustym strumieniu daje LSN=44 i EOF=76. Jednostka replay to pełna ramka z poprawnym CRC, bez osobnego commit i gwarancji rollbacku paczki bez odpowiedzi. Nowy kind/niezerowe flagi wymagają rozszerzenia formatu.

5.4 Segment danych

Jeden flush tworzy jeden segment obejmujący wszystkie niepuste szeregi zamrożonej generacji head. Jego numer jest niezależny od plików WAL. W segmencie znajdują się: 48-bajtowy nagłówek, bloki próbek, tablica indeksu oraz 4-bajtowa suma całego pliku bez tej końcowej sumy.

Offset	Rozmiar	Pole	Znaczenie
0	8 B	magic	ASCII `RSTSEG02`.
8 / 10 / 12	2 / 2 / 4 B	version / flags / header_len	`u16=2` / `u16=0` / `u32=48`.
16	8 B	generation	Numer generacji `u64`.
24	8 B	cutoff_lsn	Końcowy bajtowy LSN prefiksu objętego zrzutem.
32	8 B	series_count	Liczba bloków i wpisów indeksu.
40	8 B	index_offset	Początek tablicy indeksu od początku pliku.

Bloki następują bez przerw, w rosnącej kolejności `SeriesId`. Blok zawiera wyłącznie pary `timestamp:i64`, `value_bits:u64`. Próbki są ściśle rosnące w czasie. Identyfikator szeregu nie jest powtarzany przy każdym punkcie. Pustych bloków nie zapisuje się.

Wpis indeksu zajmuje 48 B:

Offset	Rozmiar	Pole	Znaczenie
0	8 B	series_id	Identyfikator szeregu.
8	8 B	min_time	Pierwszy czas w bloku, `i64`.
16	8 B	max_time	Ostatni czas w bloku, `i64`.
24	8 B	data_offset	Początek bloku względem początku pliku.
32	8 B	point_count	Liczba próbek w bloku.

Offset	Rozmiar	Pole	Znaczenie
40	4 B	block_crc32	CRC surowych bajtów całego bloku.
44	4 B	reserved	Zero; miejsce zastrzeżone.

Dla N punktów i K niepustych szeregów:

$$\text{index_offset} = 48 + 16N, \qquad \text{file_size} = 48 + 16N + 48K + 4.$$

Segment jednego szeregu z dwiema próbkami ma więc 132 B, offset indeksu 80 i offset danych 48. Jego wpis indeksu zajmuje bajty 80–127, a końcowe CRC bajty 128–131.

Przy uruchomieniu kontroluje się długość, sumę całego segmentu, zakresy offsetów, kolejność identyfikatorów, kolejność czasów i zgodność min/max. Indeks pozostaje w RAM; próbki są odczytywane na żądanie. Przy odczycie całego bloku można dodatkowo sprawdzić jego CRC. Odczyt wycinka korzysta z pozycyjnego odczytu i wyszukiwania binarnego; nie wymaga wczytania całego szeregu. Podczas pracy żaden inny proces nie może modyfikować plików.

5.5 Manifest i katalog szeregów

Plik `MANIFEST` jest kopertą binarną z dokumentem JSON UTF-8. Nagłówek ma 16 B: magic `RSTMAN03` (8 B), `version:u16=3`, `flags:u16=0`, `json_len:u32`. Dalej znajdują się bajty JSON i CRC-32 całego nagłówka oraz payloadu. Maksymalna długość JSON to 64 MiB; dodatkowe bajty po CRC są błędem.

Przykład dotyczy pierwszej paczki dwóch punktów z API (sekcja 8): jej ramka ma 136 B, a koniec w pliku WAL z 32-bajtowym nagłówkiem wypada na pozycji 168. Wartość CRC segmentu jest miejscem na sumę wyliczaną z rzeczywistych danych.

```
{
  "format_version": 3,
  "generation": 1,
  "checkpoint_lsn": 136,
  "replay": {"segment_id": 1, "offset": 168},
  "next_series_id": 2,
  "segments": [
    {
      "generation": 1,
      "file": "segment-00000000000000000001.bin",
      "cutoff_lsn": 136,
      "size_bytes": 132,
      "crc32": 0
    }
  ],
  "series": [
    {
      "id": 1,
      "metric": "temperature_celsius",
      "labels": [["instance", "1"], ["service", "api"]],
      "sealed_through": 1792843201000
    }
  ]
}
```

`generation` numeruje head, nie WAL. Checkpoint C to koniec wszystkich rekordów odwzorowanych w segmentach; replay (a, p) wskazuje koniec ostatniej z nich w zachowanym pliku-kotwicy, również przy p =EOF:

$$C = base_lsn + (replay.offset - 32).$$

Kursor musi być granicą pełnej ramki lub początkiem pustego strumienia.

Katalog i S_s pochodzą z chwili freeze, bez późniejszych TS/active (ich definicje i próbki są w WAL). Tablice są rosnące po ID/generacji; nazwa segmentu to pojedynczy komponent ścieżki, bez wyjścia z katalogu. CRC wpisu jest końcowym CRC pliku. Pola JSON mogą mieć dowolną kolejność: CRC obejmuje zapisane bajty, a liczniki dekoduje się całkowitoliczbowo, bez f64.

Początek: generation=0, $C = 0$, replay=(1,32), next_series_id=1, puste tablice. Następny LSN wynika z końca replay, ID przekracza wszystkie widziane identyfikatory; GC/aliasowanie poza MVP. Walidacja: wersja, CRC, kursor, rozmiary/sumy segmentów, rosnące generacje, unikalne ID i rozłączne zakresy bloków TS. Każdy TS katalogu ma blok; `sealed_through` to jego największy czas na dysku, przed replay $H_s = S_s$.

5.6 Indeksy w pamięci i plan odczytu

TSMap (rys. 4) używa odwróconych, uporządkowanych list ID dla metryki i par etykiet. Wybór zaczyna od metryki, przecina listy matcherów i sprawdza pełny klucz. Pusty matcher wybiera brak etykiety; pustych wartości nie trzeba indeksować.

Deskryptory TS zawierają segment, min/max, offset i liczbę punktów. Reguła $t > S_s$ zapewnia rozłączne zakresy: deskryptory eliminują zbędne segmenty, binarne wyszukiwanie wybiera wycinki próbek. Indeksy odbudowuje silnik, bez zewnętrznej bazy. Ich RAM zależy od liczby TS, długości etykiet i segmentów; bez kompaktowania liczba deskryptorów rośnie.

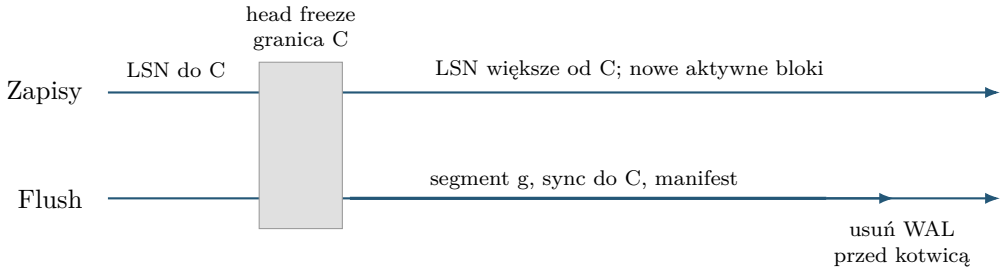
6 Checkpoint i odzyskiwanie po awarii

6.1 Trwałe opublikowanie generacji

Generacja head g i numer WAL są niezależne. Działa najwyżej jeden flush danych. Kolejność publikacji jest następująca:

Faza	Operacje w kolejności
Bariera head	Uzyskać wyłączną barierę, wstrzymując nowe części pamięciowe. Dokończyć rozpoczęte enqueue–head append (bez czekania na waiterów). Ustalić $C = enqueued_end$: wszystkie ramki do C są już w head; sam Written/Synced tego nie gwarantuje. Zapamiętać katalog i następny ID. Pod blokadami TS przenieść aktywne wektory do frozen g, podnieść S_s i utworzyć active $g + 1$. Zwolnić barierę: nowe zapisy mają LSN $> C$. Bez sync i rotacji WAL.
Segment w tle	Zapisać `segments/segment-<g>.bin.tmp` z cutoff C; sync pliku → rename → sync katalogu segments/.
Kotwica WAL	Poza blokadami TS wykonać `wal.wait_for(C,Synced)`, również dla danych None. Trwała kotwica to plik a z ostatnią ramką do C i fizyczny koniec p.
Manifest	Zapisać/synchronizować `MANIFEST.tmp`: segmenty, katalog *z chwili zamrożenia*, g, C, replay (a, p). Atomowo zastąpić MANIFEST i sync katalogu danych. To punkt publikacji checkpointu.
Podmiana head	Pod krótkimi blokadami TS zastąpić frozen g deskryptorami dyskowymi. Nie zmieniać nowszych active, H_s, last_lsn. ReadView zachowują stare uchwyty.
Sprzątanie	Usunąć wyłącznie WAL o ID $< a$, zamknięte (potwierdza flusher) i z końcem $\leq C$; zachować kotwicę a, wszystkie późniejsze i aktywny plik. Sync wal/, zwolnić własne uchwyty RAM, zakończyć flush.

Kotwica pozostaje także przy p =EOF. Koszt jednego dodatkowego pliku eliminuje kopiowanie/obcinanie początku aktywnego WAL.



Rysunek 8: Bariera dotyczy tylko stanu head. Rotacja WAL i synchronizacja trwałości nie są częścią tej pauzy.

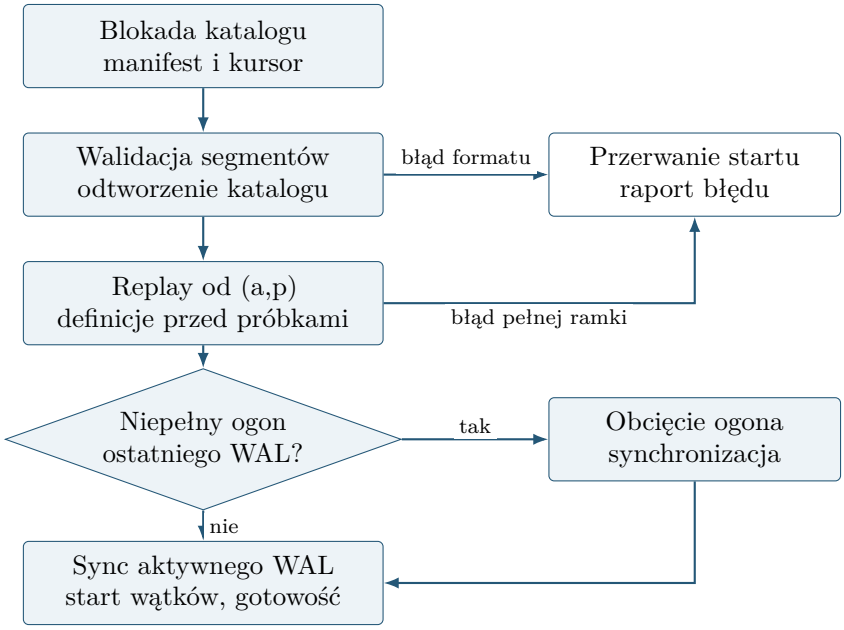
System docelowy to Linux z lokalnym systemem plików obsługującym atomowy rename oraz synchronizację plików i katalogów. Written nie gwarantuje przetrwania awarii hosta. Synced oraz checkpoint opierają się na poprawnych gwarancjach fdatasync/fsync; nie obejmują fizycznego uszkodzenia nośnika.

6.2 Algorytm uruchomienia

1. Sprawdzić konfigurację i uzyskać LOCK; ready=false do końca recovery. Pustą bazę inicjalizować trwale: WAL 1 z base=0, potem manifest z kursorem (1,32). Przerwaną inicjalizację wolno dokończyć tylko przy poprawnym pustym WAL, bez danych i opublikowanych segmentów. Inny brak manifestu oznacza błąd, bez resetu.
2. Zweryfikować manifest i jego segmenty; odbudować TSMap, indeksy, ID, granice zamknięcia. Początkowo `last_lsn=C`. Segmentów spoza manifestu nie dołączać.
3. Z kursora (a, p) sprawdzić istnienie kotwicy, nagłówek, offset i zgodność z C; zebrać WAL $a, \ldots, h$ bez luk. Czytać od p, kolejne pliki od ich nagłówków. Kontrolować długości, kind, flagi, CRC i ciągłość base_lsn z końcem poprzednika. Bajty przed kursorem pokrywa checkpoint.
4. Każdą ramkę zastosować w całości: definicje przed próbkami; zaktualizować last_lsn. Nieznany ID lub ten sam klucz pod innym ID jest błędem, bez GC, aliasów i pomijania. Następny ID musi przekraczać wszystkie zaobserwowane.
5. Obciąć można tylko niepełny nagłówek ramki/payload na końcu ostatniego WAL h. Pełna ramka z błędnym CRC, nieznany kind, luka i uszkodzenie wcześniejszego pliku zatrzymują start.
6. Zsynchronizować odtworzony/naprawiony plik h; wszystkie trzy granice WAL ustawić na koniec replay (starsze pliki zsynchronizowano przy rotacji). Tylko h otworzyć do dopisywania. Odtworzone próbki umieścić w active generacji $g + 1$.
7. Po poprawnej walidacji usunąć .tmp, sieroty segmentów i zbędne WAL sprzed kotwicy; zachować $a, \ldots, h$ do kolejnego checkpointu. Uruchomić flusher/syncer i ogłosić gotowość.

Replay nie ponawia testów bieżącego zegara ani zmienionego W, więc nie odrzuca dawnych zapisów. Nie utrwala się pierwotnego pobytu w RAM: odzyskane próbki od razu kwalifikują się do czasowego flush. Za mały nowy limit pamięci zatrzymuje start z opisem problemu.

6.3 Scenariusze awarii i granice gwarancji

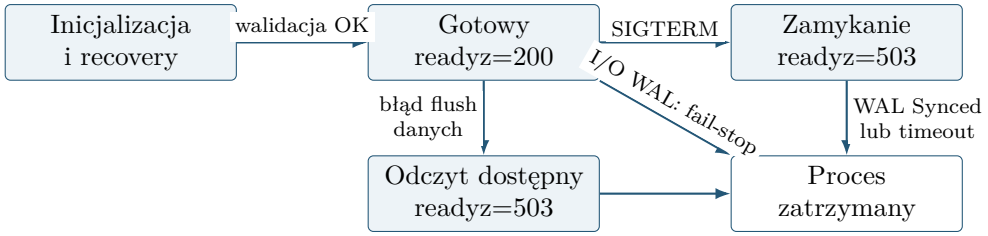


Rysunek 9: Replay rozpoczyna się od trwałego kursora checkpointu. Uszkodzona pełna ramka nie jest traktowana jak niepełny ogon.

Moment awarii	Stan po restarcie	Znaczenie dla klienta
Przed enqueue	Brak nowej ramki.	Brak sukcesu żądania.
Po enqueue, przed head append	Ramka może pozostać tylko w RAM albo być już zapisana.	Bez odpowiedzi; replay może zastosować całość.
Po odpowiedzi None, przed write	Paczka może zniknąć.	To dopuszczona słabsza trwałość, nie błąd recovery.
Po Written, przed Synced	Przy awarii procesu ramka wraca z WAL.	Awaria hosta może utracić niesynchronizowane dane.
Po Synced	Ramka pochodzi z WAL lub checkpointu.	Potwierdzone dane przetrwają w granicach gwarancji nośnika i systemu plików.
W środku końcowej ramki	Niepełny ogon ostatniego pliku jest odcięty.	Brak potwierdzenia Written/Synced dla tej ramki.
Podczas zapisu segmentu danych	Obowiązuje stary manifest i jego kursor.	Replay obejmuje także zapisy przyjęte podczas flush.
Po rename segmentu, przed manifestem	Segment jest sierotą; stary checkpoint pozostaje autorytatywny.	Dane nie są odtwarzane podwójnie.
Po manifest, przed usunięciem WAL	Replay zaczyna się od nowego kursora.	Stare pliki nie są ponownie stosowane.
Podczas usuwania starych WAL	Kotwica i późniejsze pliki pozostają.	Zapisy po C nadal można odtworzyć.

Błąd zapisu segmentu danych zatrzymuje nowe przyjęcia, zachowując head i WAL do diagnostyki lub restartu. Błąd I/O samego WAL jest polityką fail-stop i kończy proces. Granice zamknięcia z nieopublikowanego zrzutu mogą po restarcie wrócić do poprzedniego checkpointu; nie narusza to gwarancji Written/Synced.

6.4 Cykl życia procesu



Rysunek 10: Diagram stanów procesu. Błąd startu kończy proces bez gotowości; ze stanu ograniczonego odczytu operator może wykonać restart/zamknięcie.

SIGTERM zamyka przyjęcia i kończy pamięciowe części zadań, następnie wymusza Written i Synced do końcowego enqueued_end, również dla None. Dopiero potem zatrzymuje wątki WAL. Flush danych kończy lub bezpiecznie przerywa, zachowując pliki replay; pełny flush head nie jest wymagany. Po 30 s możliwe jest wymuszone zatrzymanie i zwykłe gwarancje wybranego trybu trwałości.

7 Język i wykonywanie zapytań

7.1 Obsługiwana składnia

Obsługiwany jest nazwany selektor z dokładnymi matcherami, jedna funkcja czasowa na szereg oraz skalar (np. 1+1 do próby połączenia). Gramatyka:

```
expr       = selector | function "(" selector "[" duration "]" ")"
           | scalar ;
selector   = metric [ "{" [ matcher { "," matcher } ] "}" ] ;
matcher    = label "=" string ;
function   = "avg_over_time" | "min_over_time" | "max_over_time"
           | "sum_over_time" | "count_over_time" ;
duration   = positive_integer unit ;
unit       = "ms" | "s" | "m" | "h" | "d" | "w" ;
scalar     = number [ "+" number ] ;
```

Nazwy: sekcja 3; napisy w podwójnych cudzysłowach z escape JSON; number to skończone f64 o składni JSON. Whitespace między tokenami jest ignorowany, parser zużywa całe wejście. Powtórzony matcher i etykieta z prefiksem `__` są błędem; nazwę metryki podaje się wprost. Maksimum tekstu: 16 KiB.

```
temperature_celsius
temperature_celsius{service="api",instance="1"}
avg_over_time(temperature_celsius{service="api"}[5m])
count_over_time(temperature_celsius[30s])
1+1
```

Odrzucane: regex, matchery inne niż =, selektor bez nazwy, samodzielny range vector, podzapytania, offset, @, zagnieżdżenia i agregacje między TS (np. `sum(temperature_celsius)`). Tokenizator i parser rekurencyjny tworzą AST, bez generatora pełnego PromQL. Walidacja sprawdza nazwy, dodatnie/mieszczące się okno i typ wyniku; błąd zawiera offset bajtowy i opis.

7.2 Czas obliczenia i wybór próbek

Instant oblicza wyrażenie w T, range na siatce

$$T_k = start + k \cdot step, \qquad k = 0, \ldots, \lfloor (end - start)/step \rfloor .$$

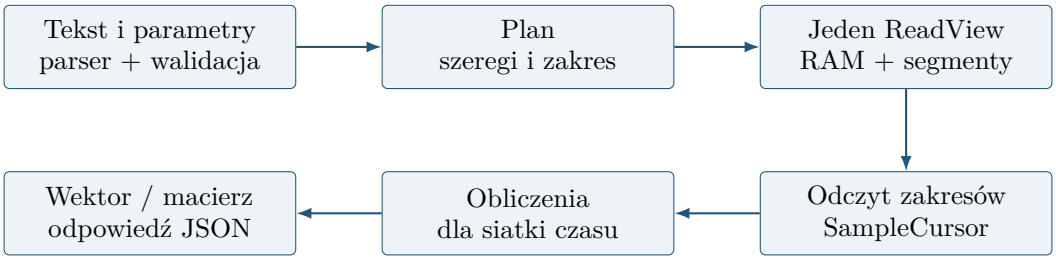
Wymagania: $step > 0$, $start \leq end$, sprawdzana arytmetyka całkowitych milisekund. Koniec wchodzi do wyniku tylko na siatce.

Wyrażenie	Semantyka dla każdego pasującego szeregu
Selektor	Najnowsza próbka w $[T - D, T]$, domyślnie $D = 5$ min (lookback). Bez interpolacji, agregacji i markerów staleness; brak próbki pomija TS. Timestamp wyniku to T, nie czas pomiaru; zachowuje `__name__`.
Funkcja z R	Okno $(T - R, T]$: lewa granica wykluczona, prawa włączona [4]; lookback nie ogranicza okna. Zachowuje etykiety, usuwa nazwę metryki. Puste okno pomija TS dla wszystkich funkcji, także count (brak danych nie oznacza zera).
`avg_over_time`	Średnia arytmetyczna, równa waga próbek.
`min_over_time`, `max_over_time`	Minimum / maksimum.
`sum_over_time`	Suma w chronologicznym porządku.
`count_over_tim e`	Liczba próbek jako wartość liczbowa.

Funkcje dotyczą próbek liczbowych [5]. Obliczenia f64 używają sumowania kompensowanego; średnia = suma/liczność. Przy przepełnieniu sumy średnią przelicza się ze składników x_i/n. NaN i nieskończoności wyników koduje się jako napisy JSON.

Przykład granic: punkty (czas w s, wartość): $(0, 10), (60, 20), (120, 30), (180, 40)$. Dla `avg_over_time(metric[2m])`: $T = 120 \Rightarrow (0, 120] \Rightarrow 25$; $T = 180 \Rightarrow (60, 180] \Rightarrow 35$; $T = 300 \Rightarrow (180, 300] \Rightarrow$ brak elementu. Zakres $[120, 190]$, krok 60 s oblicza tylko 120 i 180. Przeniesienie próbek do segmentu nie zmienia wyniku.

7.3 Plan wykonania i limity pracy



Rysunek 11: Przepływ zapytania. Każdy etap może zakończyć się błędem lub przekroczeniem budżetu.

Plan pobiera $[start - D, end]$ dla selektora lub $(start - R, end]$ dla funkcji, z bezpiecznym przycięciem do dziedziny czasu. ReadView raz kopiuje potrzebne active, współdzieli frozen; dysk czyta blokowo (I/O 64 KiB). Przesuwane okno utrzymuje sumę/liczność, a min/max kolejki monotoniczne, bez ponownego odczytu całego zakresu w każdym kroku. Nieskończony stan sumy odbudowuje się z bieżącego okna, aby skrajna wartość po jego opuszczeniu nie zatruwała następnych wyników.

Budżet zapytania: 1000 TS, 2 mln różnych wejściowych próbek, 1 mln punktów wyniku łącznie, 64 MiB i 30 s. RAM obejmuje metadane widoku, kopie active, okna i wynik przed serializacją; wspólne frozen rozlicza globalny budżet head, także podtrzymywane tylko przez ReadView. Odczyt strumieniowy nie znosi limitu próbek. Przekroczenie oznacza błąd bez częściowego wyniku; timeout sprawdza się też w pętlach obliczeń/I/O, rozłączenie sygnalizuje anulowanie.

Wyniki są leksykograficzne po kanonicznym kluczu, punkty chronologiczne. Wartości f64 to napisy, timestampy liczby sekund Unix. Instant scalar to para czasu/wartości; range scalar to macierz jednego szeregu o pustych etykietach.

8 Komunikacja i publiczne API

8.1 Transport i konwencje

HTTP/1.1 z keep-alive, UTF-8, JSON `application/json`; bez WebSocketów, brokera i skompresowanych żądań. Profil zapytań przejmuje endpointy, kopertę, wektory i macierze Prometheusa [3]; nie zapewnia pełnego PromQL/Prometheusa. Własny zapis nie jest Remote Write ani OpenMetrics.

8.2 Zapis pojedynczy i paczkowy

`POST/api/v1/write` przyjmuje dokładnie jeden z dwóch wariantów: obiekt pojedynczego pomiaru albo obiekt zawierający wyłącznie niepustą tablicę `points`. Każdy pomiar wymaga pól `metric`, `timestamp` i `value`; brak `labels` oznacza pusty zbiór. `timestamp` jest całkowitą liczbą **milisekund**, a `value` liczbą JSON. W jednej paczce wolno przesłać do 10 000 punktów. Maksymalny rozmiar surowego żądania wynosi 4 MiB. Opcjonalny parametr query `durability=none|written|synced` nadpisuje ustawienie procesu; domyślnie jest to written. Powtórzony parametr, nieznana nazwa albo wartość jest błędem 400. Tryb dotyczy całej paczki i nie zmienia jej atomowości.

```
POST /api/v1/write
Content-Type: application/json

{
  "metric": "temperature_celsius",
  "labels": {"service": "api", "instance": "1"},
  "timestamp": 1792843200000,
  "value": 24.5
}
```

Wariant paczkowy:

```
{
  "points": [
    {
      "metric": "temperature_celsius",
      "labels": {"service": "api", "instance": "1"},
      "timestamp": 1792843200000,
      "value": 24.5
    },
    {
      "metric": "temperature_celsius",
      "labels": {"service": "api", "instance": "1"},
      "timestamp": 1792843201000,
      "value": 25.5
    }
  ]
}
```

Odpowiedź dla pierwszej paczki dwóch nowych punktów w pustej bazie:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "status": "success",
  "data": {"inserted": 2, "duplicates": 0, "lsn": 136, "durability": "written"}
}
```

`inserted+duplicates` = liczba punktów wejścia. LSN to koniec ramki; durability potwierdza osiągnięty tryb. Gwarancje określa sekcja 5.2, a reguły duplikatów i atomowości sekcja 3. Przed enqueue błąd odrzuca całą paczkę; po enqueue timeout/anulowanie/ utrata połączenia nie wycofują zmian. Timeout daje 503/timeout z niepewnym wynikiem: WAL może osiągnąć wymagany poziom później.

8.3 Zapytania chwilowe i zakresowe

Obsługiwane są metody GET oraz POST z `application/x-www-form-urlencoded`. Nie wysyła się JSON jako ciała zapytania Prometheus. Parametry w danym żądaniu pochodzą albo z query string, albo z formularza; duplikaty parametrów i mieszanie obu źródeł są błędem. Całość parametrów nie może przekraczać 32 KiB.

Endpoint	Parametry	Znaczenie
`/api/v1/query`	`query`; opcjonalnie `time`, `timeout`	`time` domyślnie równe chwili odebrania żądania.
`/api/v1/query_range`	`query`, `start`, `end`, `step`; opcjonalnie `timeout`	Obliczenie na siatce czasu; wszystkie cztery główne parametry są wymagane.

Czasy przyjmują sekundy Unix z częścią ułamkową lub RFC 3339 ze strefą. Po konwersji muszą mieścić się w zakresie czasu z sekcji 3. Wymagana jest dokładność co najwyżej milisekundowa; dodatkowe niezerowe cyfry są odrzucane zamiast cichego zaokrąglania. Krok przyjmuje dodatnią liczbę sekund, np. `0.5`, albo pojedynczą jednostkę jak `15s`. Okno i timeout przyjmują dodatnią liczbę całkowitą z jedną jednostką z gramatyki `duration`. Timeout efektywny jest mniejszą wartością limitu serwera i parametru klienta.

Nieznane parametry zwracają 400; nie są ignorowane. Parametry opcjonalne pełnego API Prometheusa, np. `lookback_delta` i `limit`, pozostają nieobsługiwane w tym profilu. Grafana korzysta z konfiguracji nieprzesyłającej tych rozszerzeń.

```
GET /api/v1/query?query=temperature_celsius&time=1792843201

{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {"__name__": "temperature_celsius", "service": "api", "instance": "1"},
        "value": [1792843201, "25.5"]
      }
    ]
  }
}
```

Przykład odpowiedzi zakresowej dla tej samej metryki od 1792843200 do 1792843201 z krokiem 1 s:

```
{
  "status": "success",
  "data": {
    "resultType": "matrix",
    "result": [
      {
        "metric": {"__name__": "temperature_celsius", "service": "api", "instance": "1"},
        "values": [[1792843200, "24.5"], [1792843201, "25.5"]]
      }
    ]
  }
}
```

Brak danych oznacza `result:[]` z HTTP 200. Wynik `1+1` w zapytaniu chwilowym ma `resultType:"scalar"` oraz `result:[1792843201,"2"]` dla powyższego czasu. Wartości specjalne wyników obliczeń koduje się jako `"NaN"`, `"+Inf"` lub `"-Inf"`. Wejście zapisu nadal ich nie dopuszcza.

8.4 Metadane i stan serwera

Metoda i ścieżka	Kontrakt profilu
GET `/api/v1/labels`	Posortowana lista nazw etykiet oraz `__name__` z katalogu.
GET `/api/v1/label/<name>/values`	Posortowane unikalne wartości etykiety; dla `__name__` nazwy metryk. Nieistniejąca poprawna nazwa daje pustą listę.
GET `/api/v1/status/buildinfo`	Koperta sukcesu z `data.version`, `data.revision`, `data.branch`, `data.buildDate` i `data.implementation`. To dane kompilacji TSDB; nie podszywają się pod wersję Prometheusa.
GET `/healthz`	200 i `ok` jako tekst, jeśli działa pętla obsługi procesu.
GET `/readyz`	200 i `ready` po recovery i przy sprawnym zapisie; 503 i tekst przy braku gotowości.

Odpowiedzi metadanych stosują kopertę `status:"success",data:[...]`. Endpointy etykiet nie przyjmują w tym profilu filtrów czasu ani `match[]`; operują na całym katalogu. Nie obsługuje się `/series`, `/metadata`, reguł, exemplarów ani administracyjnego usuwania danych. Dashboard nie wymaga tych operacji.

8.5 Błędy i przeciążenie

HTTP	errorType	Zastosowanie
400	`bad_data`	Błędny JSON, parametry, składnia, nazwy, typy albo nieobsługiwana konstrukcja języka.
404	`not_found`	Nieznany endpoint.
405	`bad_data`	Niewłaściwa metoda; nagłówek `Allow`.
409	`conflict`	Różne wartości dla tego samego czasu i szeregu.
413	`too_large`	Za duże ciało, tekst zapytania lub liczba punktów paczki.

HTTP	errorType	Zastosowanie
415	`bad_data`	Nieobsługiwany typ treści lub kodowanie wejścia.
422	`execution`	Zamknięty zakres, przekroczone okno opóźnienia, limit szeregów, pamięci lub rozmiaru wykonywanego zapytania.
503	`unavailable` / `timeout`	Pełna kolejka, zajęte miejsca zapytań, brak gotowości, timeout albo zatrzymany zapis po błędzie I/O.
500	`internal`	Nieoczekiwany błąd lub uszkodzenie podczas odczytu; szczegóły ścieżek i systemu trafiają do logów.

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{
  "status": "error",
  "errorType": "execution",
  "error": "point belongs to a sealed time range",
  "details": {"reason": "sealed_range", "point_index": 1}
}
```

`details` jest rozszerzeniem własnego endpointu zapisu; indeks jest liczony od zera w oryginalnej paczce. Dla zapytań wystarcza standardowa koperta z `status`, `errorType` i `error`. Powody odrzucenia out-of-order rozróżnia się jako `sealed_range` i `out_of_order_window`.

Globalny semafor dopuszcza do 8 żądań zapisu łącznie: dekodowanych, oczekujących i przetwarzanych. Adapter rezerwuje miejsce przed zbuforowaniem ciała. Wewnątrz limitu pula wykonuje domyślnie cztery operacje równolegle; kolejka przekazuje zadania FIFO, ale nie narzuca kolejności zakończenia rozłącznych paczek. Zajęte miejsca skutkują od razu 503 z `Retry-After:1`. Oczekiwanie na pamięć nie utrzymuje blokad potrzebnych flush. Limit ośmiu przyjęć obejmuje również żądania oczekujące na Written/Synced, choć nie zajmują już one miejsca w puli części pamięciowej. Zapytania korzystają z osobnego semafora dwóch miejsc, również bez nieograniczonego oczekiwania.

Generator ponawia 503 z wykładniczym opóźnieniem od 1 s do 30 s i losowym odchyleniem. Nie ponawia automatycznie błędów walidacji, konfliktu i zamkniętego zakresu. Po błędzie połączenia dopuszcza niepewność wyniku opisaną wcześniej.

9 Technologie, konfiguracja i środowisko

9.1 Wybór technologii

Wybór	Uzasadnienie, alternatywy i ograniczenia
Rust 2024	Kontrola danych i własności pamięci; zamiast C++/Go. Koszt nauki, konieczne własne limity alokacji.
Axum 0.8, Tokio 1	Routing, ekstraktory i runtime [8]; alternatywy: Actix Web/Hyper. Blokujące I/O poza wątkami sieci.
Serde 1, serde_json 1	Typowane DTO zamiast ręcznego JSON; własne limity i wykrywanie powtórzonych kluczy.
time 0.3, Clap 4	RFC 3339 i jeden sposób konfiguracji CLI; alternatywy: chrono/plik TOML.

Wybór	Uzasadnienie, alternatywy i ograniczenia
std::fs, własny kodek	Własny silnik jako cel projektu, jawne/testowalne formaty; zamiast RocksDB/SQLite, kosztem odpowiedzialności za recovery.
Arc, RwLock, Mutex, kanały	Blokady TS, bariera head, WAL/watch; pula pamięciowa i async wait. Bez spinlocków i map lock-free.
crc32fast 1, thiserror 2	Gotowy CRC i typowane błędy zamiast własnego algorytmu i błędów tekstowych.
tracing 0.1, tracing-subscriber 0.3	Zdarzenia i czasy operacji zamiast własnego loggera; bez pełnych treści pomiarów.
Grafana OSS 13.2.0	Wykresy, wybór czasu i provisioning [7]; zamiast własnego GUI, zgodność do sprawdzenia.
Compose v2, Linux	Sieć, usługi i wolumeny [11]; zamiast ręcznej instalacji/Kubernetes, bez orkiestracji klastra.
cargo test, Tower 0.5, generator Rust/reqwest 0.12	Testy modułów, HTTP i obciążenia w jednym stosie, stałe ziarno generatora; zamiast zewnętrznego frameworka.

To wybrane linie zgodności, bez zmiany zależności prototypu. Implementacja utrwala wersje w `Cargo.lock`, toolchain w `rust-toolchain.toml`; demo przypina `grafana/grafana:13.2.0` oraz zapisuje digest. Walidacja: Linux x86-64; inne systemy przez kontenery Linux.

9.2 Konfiguracja procesu

Źródłem konfiguracji są argumenty CLI; brak parametru oznacza wartość domyślną. Compose przekazuje je jako tablicę `command`. Nie wprowadza się równoległych zmiennych środowiskowych dla tych samych ustawień. Zmiany wymagają restartu.

Parametr	Domyślnie	Znaczenie i walidacja
`--listen`	`127.0.0.1:8080`	Adres HTTP; w kontenerze jawnie `0.0.0.0:8080`.
`--data-dir`	`./data`	Katalog dostępny do zapisu, jeden proces na katalog.
`--buffer-duration`	`30m`	Czas do zlecenia flush; dodatnia pojedyncza jednostka czasu.
`--write-workers`	4	Liczba zadań zapisu wykonywanych równolegle, od 1 do 8; nie zmienia liczby logicznych WAL.
`--wal-durability`	`written`	Wybór none, written albo synced; żądanie może nadpisać ten poziom.
`--wal-flush-interval`	`10ms`	Dodatni interwał zapisu bez oczekujących klientów; nie jest terminem gwarantowanej trwałości.
`--wal-group-delay-us`	200	Nieujemne mikrosekundy zbierania grupy po pobudzeniu przez oczekujących.
`--wal-sync-interval`	`1s`	Dodatni okres fdatasync; Synced wyzwala wcześniejszą synchronizację.

Parametr	Domyślnie	Znaczenie i walidacja
`--wal-segment-bytes`	134217728	128 MiB; co najmniej 8388640 B, aby zmieścić maksymalną ramkę i nagłówek.
`--max-wal-buffer-bytes`	33554432	32 MiB na bufor przyjmujący i przejęty przez flusher, minimum 16 MiB.
`--wal-wait-timeout`	`30s`	Dodatni timeout odpowiedzi Written/Synced; nie wycofuje head.
`--max-buffer-bytes`	536870912	512 MiB; dodatni, co najmniej 65536 B. Wartość 0 wyłącza limit próbek, lecz nie pozostałe limity.
`--out-of-order-window`	`5m`	Nieujemny, nie większy od czasu buforowania. Nadal obowiązuje $t > S_s$.
`--max-future-skew`	`60s`	Dopuszczalne wyprzedzenie zegara, nieujemne.
`--lookback-delta`	`5m`	Dodatni zakres wyboru ostatniej próbki.
`--max-series`	10000	Maksymalna liczba szeregów w katalogu; liczba dodatnia.
`--max-metadata-bytes`	67108864	64 MiB na katalog i indeksy, z uwzględnieniem kopii katalogu dla checkpointu.
`--query-timeout`	`30s`	Dodatni limit pojedynczego zapytania.
`--log-level`	`info`	Jeden z: error, warn, info, debug, trace.

Domyślne, stałe limity ochronne z poprzednich sekcji tworzą profil v1: 4 MiB wejścia, 10 000 punktów paczki, 8 przyjętych żądań zapisu, dwa zapytania, 64 MiB na zapytanie i 8 MiB całej ramki WAL. Łączny budżet surowych ciał i zdekodowanych paczek oraz przygotowanych ramek WAL wynosi 64 MiB. Obejmuje także paczki aktualnie przetwarzane. Pamięć jest rezerwowana przed dekodowaniem i przy zmianie reprezentacji; przekroczenie budżetu daje 503 przed enqueue. Osobne 32 MiB WAL obejmuje kopie już przekazane do bufora i flushera. Odczyt ciała ma timeout 10 s.

Limit metadanych sprawdza się przed rejestracją szeregu oraz przed utrwaleniem kolejnej paczki, uwzględniając miejsce na indeks następnego zrzutu. Brak miejsca odrzuca nową paczkę, zamiast przyjmować dane, których zaplanowany flush przekroczyłby limit. Operator może podnieść limit po restarcie, co zmienia założenia zużycia RAM. Wyłączenie limitu bufora nie wyłącza kontroli przepełnień ani błędów alokacji.

9.3 Sieć, trwałość i granica dostępu

Topologię, porty, DNS i montowania pokazuje rys. 2. Publikacje Compose: `127.0.0.1:3000:3000` i `127.0.0.1:8080:8080`. Baza działa bez root z prawem zapisu do własnego wolumenu. Grafana wymaga logowania; hasło administratora dostaje przez nieśledzony plik środowiskowy. API v1 nie ma auth/TLS: granicą jest lokalny host i zaufana sieć demo; wyjście poza nią wymaga osobnej warstwy dostępu. Przeglądarka nie łączy się z TSDB, więc CORS nie jest potrzebny.

Compose czeka na `/readyz` przed startem generatora. Po restarcie generator ponawia połączenie, Grafana odzyskuje odczyt pod stałym adresem. Zatrzymanie kontenerów zachowuje wolumen; jego usunięcie jest osobną operacją operatora.

9.4 Diagnostyka

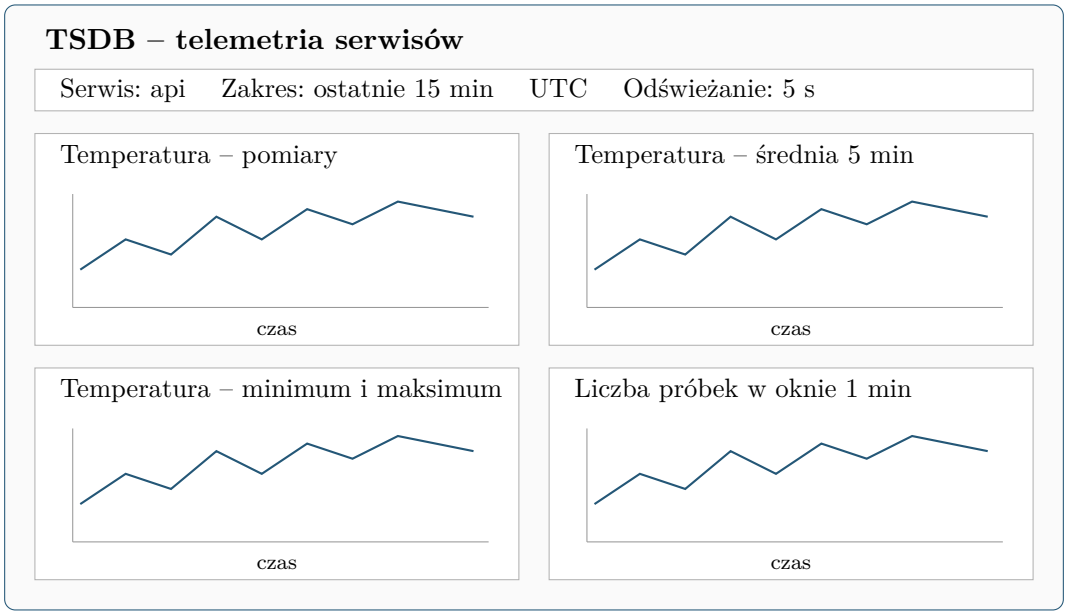
Logi zawierają identyfikator żądania, endpoint, wynik, czas obsługi, LSN, wybrany tryb trwałości i liczniki punktów. Dla zrzutu zapisuje się generację head, checkpoint i kursor replay, liczbę punktów, bajty i czas. Recovery raportuje czas, liczbę odtworzonych ramek oraz ewentualne obcięcie niepełnego ogona. Nie zapisuje się pełnych ciał żądań ani wartości etykiet w każdym logu.

Do eksperymentów zbiera się RSS z systemu operacyjnego, rozmiar bufora, pojemność obu buforów WAL, różnice enqueued/written/synced, czas write i fdatasync, długość kolejki i czas flush. Wewnętrzne liczniki mogą być raportowane okresowo w logach; nie wymagają uruchamiania kolejnego systemu monitorowania.

10 Wizja interfejsu użytkownika i demonstracja

10.1 Dashboard Grafany

Dashboard „TSDB – telemetria serwisów” zawiera wybór jednego serwisu, selektor czasu, automatyczne odświeżanie co 5 s i cztery panele. Początkowy zakres to ostatnie 15 min, strefa wyświetlania UTC. Źródła generatora mają etykiety `service` i `instance`; różne instancje są osobnymi liniami na wykresie.



Rysunek 12: Makieta układu dashboardu. Linie są ilustracją, nie wynikami eksperymentu.

Panel	Wyrażenie projektowanego dashboardu
Pomiary	`temperature_celsius{service="$service"}`
Średnia 5 min	`avg_over_time(temperature_celsius{service="$service"}[5m])`
Minimum i maksimum	Dwa zapytania: `min_over_time(temperature_celsius{service="$service"}[5m])` oraz `max_over_time(temperature_celsius{service="$service"}[5m])`.
Liczba próbek	`count_over_time(temperature_celsius{service="$service"}[1m])`

Zmienna `service` jest typu *Custom*, z wartościami `api` i `worker`. Wyłączone są opcje wielokrotnego wyboru i „All”, aby Grafana nie generowała regexów. Zapytania są zapisane w trybie *Code*.

Minimalny krok panelu to 1 s; Grafana może go zwiększać dla dłuższego zakresu. Nie używa się makra generującego funkcje spoza podzbioru.

Brak danych jest przedstawiany jako przerwa lub „No data", bez łączenia luk wypełnieniem zerami. Błąd API pozostaje widocznym błędem panelu, a nie pustym poprawnym wynikiem. Panel liczby próbek nie jest licznikiem wszystkich szeregów ani agregacją między instancjami.

10.2 Profil standardowego źródła Prometheus

Provisioning ustawia stały UID, adres bazy i metodę POST. Poniższy fragment jest projektowaną konfiguracją do użycia podczas implementacji demo; znaczenie ustawień źródła wynika z dokumentacji Grafany [6].

```
apiVersion: 1
datasources:
  - name: TSDB
    uid: tsdb
    type: prometheus
    access: proxy
    url: http://tsdb:8080
    isDefault: true
    editable: false
    jsonData:
      httpMethod: POST
      timeInterval: 1s
      manageAlerts: false
      disableRecordingRules: true
      prometheusType: Prometheus
      prometheusVersion: 2.0.0
```

`prometheusVersion:2.0.0` wybiera konserwatywny profil funkcji klienta, nie deklaruje wersji silnika TSDB. Prawdziwą wersję zwraca `buildinfo`. Wyłącza się incremental querying, exemplary i funkcje edytora wymagające nieobsługiwanych endpointów. Provisioning dashboardu wskazuje katalog plików JSON i UID `tsdb`. Brakującą funkcję Grafany weryfikuje się na przypiętym obrazie, bez deklarowania zgodności ze wszystkimi wersjami.

10.3 Scenariusz demonstracji

1. Uruchomić Compose z pustym lub istniejącym wolumenem. Poczekać na gotowość bazy i uruchomienie generatorów.
2. Generatory `api` i `worker` wysyłają co sekundę temperaturę dwóch instancji. Jeden profil celowo wprowadza opóźnione punkty.
3. Otworzyć dashboard, zmienić serwis i zakres czasu; porównać pomiary z agregacjami i liczbą próbek.
4. W profilu demonstracyjnym skrócić buforowanie do 1 min, a okno opóźnienia do 30 s. Pokazać potwierdzane zapisy i odczyty podczas flush, niezmienność wcześniejszych danych po podmianie reprezentacji oraz błąd punktu skierowanego do zamrożonego zakresu.
5. Zrestartować bazę z zachowaniem wolumenu. Pokazać wcześniejsze dane i dalszy napływ pomiarów.

Profil pokazowy z krótkim buforem nie jest profilem testu zasobów z lab1. Scenariusz awarii z SIGKILL jest wykonywany w testach kontrolowanych; sam restart demonstracyjny nie dowodzi gwarancji NFR-04.

11 Weryfikacja architektury i kryteria odbioru

11.1 Powiązanie z wymaganiami

Wymaganie	Mechanizm	Weryfikacja
FR-01	HTTP, walidacja, atomowa ramka WAL.	T-01: zapis pojedynczy i paczka; błędna paczka nie zmienia danych.
FR-02	Kanoniczny klucz i katalog szeregów.	T-02: kolejność etykiet, inne wartości, puste etykiety i kolizje hash.
FR-03	Written/Synced, bufor WAL i checkpoint.	T-03, T-10, T-16: replay potwierdzonych Written/Synced; osobna ocena None.
FR-04	Siatka czasu i jeden ReadView.	T-04: lookback, granice, krok niedzielący zakresu i puste wyniki.
FR-05	Parser ograniczonej gramatyki.	T-05: wszystkie konstrukcje, pełne zużycie wejścia, odrzucane rozszerzenia.
FR-06	Agregacje osobno dla szeregu.	T-06: pięć funkcji, granice okna, kilka szeregów, tolerancja błędu f64.
FR-07	API Prometheus i provisioning.	T-07: dashboard i test połączenia na przypiętym obrazie Grafany.
FR-08	Rotacja bloków, budżet pamięci i flush w tle.	T-08, T-14–15: timer, nowe zapisy podczas flush i pamięć utrzymywana przez odczyt.
FR-09	W, H_s, S_s.	T-09: granice równości, flush, zmiana konfiguracji i restart.
NFR-01–03	Paczki, indeksy, limity i kontrola alokacji.	P-01–03: testy HTTP, przepustowości i RSS z opisanym profilem.
NFR-04	Recovery i kolejność publikacji plików.	T-10 i P-04: wymuszone awarie i czas gotowości.
NFR-05	Typowane błędy i walidacja.	T-11: macierz błędnych żądań bez zakończenia procesu.
DA-02–04, DA-08	TSMap, blokady TS, bariera rotacji i jeden WAL.	T-12–15 i P-05: równoległość, brak zakleszczeń i poprawność widoków.
DA-07, DA-09	Flusher, syncer, granice prefiksów i tryby per żądanie.	T-16–20 i P-06: przyczynowość definicji, rotacja, checkpoint i błędy I/O.

11.2 Scenariusze poprawności

T-01: Jeden punkt, paczka wieloszeregowa i 10 000 punktów: odczyt po sukcesie widzi całość. Błędny punkt nie zmienia próbek/katalogu; niewidoczne luki ID dozwolone.

T-02: Kolejność etykiet nie zmienia TS, ich wartość zmienia. Puste są usuwane; powtórzone klucze JSON i nazwy zastrzeżone odrzucane; kolizja hash nie utożsamia różnych kluczy.

T-03: Restart z samym WAL, samymi segmentami i pustym active WAL oraz stanem mieszanym; zgodne próbki, etykiety, następne ID i S_s.

T-04: Instant/range: start=end, krok=0, odwrócony zakres, koniec poza siatką, dokładna granica lookback, wiele TS i brak danych.

T-05: Parser: wszystkie funkcje, escape, whitespace, 1+1; błędna jednostka, trailing input, regex i nieobsługiwana funkcja.

T-06: Ręcznie obliczone wyniki, puste i brzegowe okno $(T - R, T]$, skrajne skończone wejścia/nieskończony wynik; RAM, dysk i mieszany widok dają to samo z tolerancją $10^{-9} \max(1, |wynik_{\text{ref}}|)$.

T-07: Przypięta Grafana: provisioning, żądania wszystkich paneli, wybór serwisu/czasu, przerwa danych, restart i test połączenia.

T-08: Flush czasowy/pamięciowy podczas odczytu: próbka raz, bez pełnej kopii RAM; za mały budżet paczki bez pętli flush.

T-09: Równość i obie strony $H - W$, S, duplikaty/konflikty; ponowna walidacja po flush. Replay po zmianie konfiguracji zachowuje dane.

T-10: ≥ 20 awarii procesu *w każdym* Written/Synced, osobny dziennik potwierdzeń klienta. Awarie: środek ramki, po write/sync, rename segmentu/manifestu, przed/po rotacji WAL i usunięciu starych plików. Każda potwierdzona paczka wraca; None i paczka bez odpowiedzi mogą wrócić w całości lub być nieobecne.

T-11: Błędne typy, NaN, za duże ciało, nieznane parametry, pełne przyjęcia, zajęte zapytania, timeout WAL: właściwy kod/opis i nadal dostępna diagnostyka.

T-12: ≥ 2 zapisy rozłącznych TS. Zatrzymać syncer po publikacji Synced: inny None (także do tego TS) i odczyt mogą się zakończyć. Konkurencyjne duplikaty/konflikty wspólnego TS mają ustaloną kolejność, bez utraty próbek.

T-13: Paczki tych samych TS w odwrotnej kolejności wejścia nie zakleszczają się; odczyt widzi całość albo stan sprzed paczki. Równoległa rejestracja klucza tworzy jeden TS; zmiana wersji katalogu w trakcie snapshot wymusza ponowienie wyboru.

T-14: Wstrzymać flush po freeze, przed trwałym segmentem: nowe zapisy działają, obie generacje są czytelne, publikacja nie zmienia wyniku i nie usuwa nowego WAL. Restart przed/po manifeście, również z nowym TS zarejestrowanym podczas flush.

T-15: Wolny flush, active bliski limitu, ReadView podtrzymuje stary blok po publikacji: pamięć rozliczona do ostatniego uchwytu; 503 przed enqueue, bez kolejnych frozen i blokowania zwalniania RAM.

T-16: Zatrzymać flusher/checkpoint: None odpowiada, Written/Synced czekają mimo widoczności head. Write zwalnia Written, fdatasync Synced; awaria przed write może utracić None. Ponowienie duplikatu None jako Synced rzeczywiście czeka.

T-17: Wstrzymać rejestrację przed enqueue definicji: konkurent nie zaloguje jej próbek wcześniej. Po zwolnieniu drugi zapis Synced, restart: definicja i oba punkty wracają. Nieznany ID zatrzymuje replay.

T-18: Syncer zapamiętuje cel, potem dopisano nowsze ramki: pierwszy sync potwierdza tylko cel. Powtórzyć przy rotacji, krótkim write, anulowaniu i rejestracji waitera po wcześniejszej zmianie watch.

T-19: WAL gotowy, head append wstrzymany: checkpoint czeka na publikację, bez czekania na optional wait. Replay ze środka/EOF, kotwica, active i LSN $> C$ zachowane. Wiele rotacji bez flush i wiele flush bez rotacji nie zmienia odczytu.

T-20: Pełny aktywny/przejęty bufor WAL: odrzucenie przed enqueue, bez części paczki i nieograniczonej kolejki. Błąd write/fdatasync kończy proces, bez fałszywego sukcesu; wcześniejsze potwierdzenia wracają przy sprawnym nośniku testowym.

Kodeki: znane bajty/długości/CRC, każde ucięcie małej ramki, nadmierne liczniki, offset poza plikiem, uszkodzony nagłówek i pełna ramka z błędnym CRC (ostatnia zatrzymuje start). Bariery testowe wymuszają przeploty; sprawdza się też ciągłość LSN między plikami, pusty nowy WAL,

niepełny ogon tylko ostatniego pliku i replay po kursorze bajtowym przy niezależnych generacjach head.

11.3 Profil pomiarów wydajności i zasobów

Raport podaje CPU/rdzenie, RAM, dysk/system plików, OS, toolchain, commit, konfigurację i współbieżność klienta. Referencja: Linux x86-64, ≥ 4 rdzenie, 8 GiB RAM, lokalny SSD bez innego obciążenia, build release; parametry nie zastępują pomiaru.

Test	Przebieg	Cel z lab1
P-01	Po 30 s rozgrzewki co najmniej 5 min pomiaru. Osobne scenariusze zapisu jednego punktu, paczki 1000 punktów, odczytu 1000 wyników i agregacji. Profil bazowy Written; czas HTTP obejmuje kolejkę, publikację head i oczekiwanie na write.	p95 zapisu punktu $\leq$ 100 ms; paczki $\leq$ 500 ms; odczytu $\leq$ 200 ms; agregacji z RAM $\leq$ 500 ms.
P-02	1000 szeregów o stałych etykietach, paczki po 1000 punktów, 4 współbieżnych klientów, Written. Co najmniej godzina, by objąć flush; raportować także najgorsze okna 1 s i 10 s.	Średnio $\geq$ 10 000 nowych potwierdzonych punktów/s, z uwzględnieniem rotacji, działania flush i błędów.
P-03	10 000 punktów/s, 1000 szeregów, buforowanie 30 min, domyślne limity, minimum 65 min. Bazowy scenariusz bez zapytań, dodatkowo wariant z dwoma zapytaniami.	RSS procesu $\leq$ 1 GiB, w tym szczyt przy flush i realokacji. Raportować rzeczywisty czas przebywania punktów w RAM.
P-04	Milion potwierdzonych punktów; osobno stan z przewagą WAL i stan po flush. Pomiar od startu procesu do pierwszego 200 z readyz; osobno cache zimny i ciepły.	Gotowość w $\leq$ 10 s i brak utraty danych w T-10.
P-05	Warianty 1, 2, 4 i 8 zadań zapisu i klientów. Osobno rozłączne szeregi oraz wszystkie paczki kierowane do jednego TS. Pomiar obejmuje co najmniej jeden flush.	Ocena rzeczywistej równoległości, kosztu blokad i wspólnego WAL; bez założenia liniowego przyspieszenia.
P-06	Osobne przebiegi None, Written i Synced dla tych samych paczek. Zmiana opóźnienia grupy i obserwacja opóźnień flushera/syncera.	Raportować p95, przepustowość i bajty oczekujące oddzielnie; None nie zalicza kryterium trwałej przepustowości.

P-01: selektor jednego TS, 1000 chwil co 1 s i próbki co 1 s; agregacja tego TS, okno 5 min, ta sama siatka, potrzebne dane w RAM. P-02/03: 1000 TS po 10 punktów/s, najwyżej cztery krótkie etykiety; bazowo 0% out-of-order, dodatkowo 1%/10% z opóźnieniem do 5 min. Odrzuceń po zamknięciu nie wlicza się do przepustowości.

Surowe próbki z 30 min: $10\,000 \cdot 1800 = 18$ mln, $18\,000\,000 \cdot 16 = 288\,000\,000$ B $\approx$ 274,7 MiB. Dochodzi zapas pojemności, active/frozen, uchwyty, metadane, przyjęcia, scratch, HTTP i zapytania. Limit head 512 MiB daje margines, **nie dowodzi RSS $\leq$ 1 GiB**. Mierzy się też wspólny WAL, oczekiwanie na TS, barierę i osobno dyskowy flush (bez globalnej pauzy). Nieosiągnięty cel raportuje się z wynikiem i wąskim gardłem.

11.4 Ryzyka i warunki akceptacji projektu

Ryzyko	Sposób ograniczenia	Obserwowany wskaźnik
Flush zużywa zapas pamięci	Rotacja przy 80% budżetu, jeden flush w tle, rozliczanie bloków podtrzymywanych przez odczyty i backpressure.	Czas flush i bariery, pamięć obu generacji, 503 i p95 zapisu.
Konkurencja o blokady i WAL	Krótkie blokady TS, enqueue bez I/O i oczekiwanie na Written/Synced dopiero po publikacji.	T-12–13, P-05–06, czas części pamięciowej i opcjonalnego oczekiwania.
Koszt out-of-order	Wektor uporządkowany, jawny limit W i zamknięta granica; test udziału opóźnionych danych.	Czas wstawienia i liczba przesuniętych próbek.
Wzrost indeksów i plików	Limit katalogu i metadanych; jawny brak retencji i kompaktowania w v1.	Liczba szeregów, segmentów, bajty metadanych i dysku.
Niepełna zgodność Grafany	Ustalony obraz, tryb Code, ograniczone zmienne i wczesny test rzeczywistych żądań.	T-07 oraz lista wywołanych endpointów i parametrów.
Błąd checkpointu	Manifest jako jedyne źródło prawdy, sync przed rename, WAL usuwany na końcu.	T-10 i testy uszkodzeń plików.

Akceptacja projektu architektury wymaga spójności kontraktów i formatów, pokrycia wszystkich obszarów wytycznych lab2 oraz prześledzenia scenariuszy awarii. Akceptacja późniejszej implementacji wymaga wykonania testów i pomiarów; niniejszy dokument nie oznacza ich zaliczenia.

A Słownik

Pojęcie	Znaczenie
Head / TSMap / TS	Pamięciowy katalog i aktywne/zamrożone próbki / jeden katalog kluczy i uchwytów z indeksem etykiet / szereg z osobną blokadą, active i referencjami do frozen/dysku.
WAL / LSN	Uporządkowany bufor przed zmianą head, trwałość zależna od trybu / koniec ramki w strumieniu bajtów bez nagłówków plików.
Flusher / Syncer	Zapisuje bufor do OS i publikuje Written / synchronizuje prefiks i publikuje Synced.
Generacja / Segment	Próbki wspólnie zamrażane (numer niezależny od WAL) / niezmienny plik próbek i indeksów powstały przez flush.
Manifest / Checkpoint	Spis opublikowanych segmentów, katalog i checkpoint / największy LSN, którego wszystkie dane są w tych segmentach.
Zamrożenie / Flush	Krótka bariera kończąca pamięciowe części zapisów / zapis frozen do segmentu i publikacja manifestu w tle.
Rotacja WAL / Kotwica	Zamknięcie pełnego pliku i otwarcie następnego, niezależnie od head / zachowany plik i offset wznowienia replay.

Pojęcie	Znaczenie
Out-of-order / S_s	Próbka starsza od najnowszej / granica zamknięcia: nowy czas musi być większy.
Lookback / Provisioning / RSS	Maksymalny wiek próbki selektora / deklaratywne źródło i dashboard Grafany / rezydentna pamięć procesu, szersza niż surowe próbki.

B Budowanie dokumentu

Źródło jest samodzielnym plikiem LaTeX. Diagramy i makieta są zapisane w TikZ, bez zależności od zewnętrznych obrazów lub usług. Wymagane są pdfLaTeX, latexmk oraz pakiety użyte w preambule, w szczególności obsługa języka polskiego i TikZ. Z katalogu głównego repozytorium:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error \
  -outdir=target/lab2-doc \
  pz/lab2/lab2-Blonowski-Zaborski.tex
```

Pliki pomocnicze trafiają do ignorowanego katalogu `target/lab2-doc`. PDF do oddania jest kopią wyniku kompilacji umieszczoną obok źródła. Po zmianach należy sprawdzić ostrzeżenia składu, odwołania, spis treści oraz czytelność tabel i diagramów.

Literatura

[1] H. Błonowski, J. Zaborski, *Etap 1: Specyfikacja wymagań i zakres pracy*, wersja 1.1, 21.10.2026, dokument projektu w `pz/lab1`.

[2] T. Pelkonen, S. Franklin, P. Cavallaro, Q. Huang, J. Meza, J. Teller, K. Veeraraghavan, *Gorilla: A Fast, Scalable, In-Memory Time Series Database*, Proceedings of the VLDB Endowment, 8(12), s. 1816–1827, 2015, sekcje 4.2–4.3, `https://www.vldb.org/pvldb/vol8/p1816-teller.pdf`.

[3] Prometheus, *HTTP API*, `https://prometheus.io/docs/prometheus/latest/querying/api/`.

[4] Prometheus, *Querying basics*, `https://prometheus.io/docs/prometheus/latest/querying/basics/`.

[5] Prometheus, *Query functions*, `https://prometheus.io/docs/prometheus/latest/querying/functions/`.

[6] Grafana Labs, *Configure the Prometheus data source*, `https://grafana.com/docs/grafana/latest/datasources/prometheus/configure/`.

[7] Grafana Labs, *Provision Grafana*, `https://grafana.com/docs/grafana/latest/administration/provisioning/`.

[8] Projekt Axum, *Dokumentacja biblioteki*, `https://docs.rs/axum/latest/axum/`.

[9] Projekt Rust, *std::fs::File*, `https://doc.rust-lang.org/std/fs/struct.File.html`.

[10] Projekt crc32fast, *Dokumentacja biblioteki*, `https://docs.rs/crc32fast/latest/crc32fast/`.

[11] Docker, *How Compose works*, `https://docs.docker.com/compose/intro/compose-application-model/`.

[12] Linux man-pages, *write(2)*, `https://man7.org/linux/man-pages/man2/write.2.html`.

[13] Linux man-pages, *fsync(2), fdatasync(2)*, `https://man7.org/linux/man-pages/man2/fsync.2.html`.

[14] Linux man-pages, *sync_file_range(2)*, `https://man7.org/linux/man-pages/man2/sync_file_range.2.html`.

[15] Projekt Tokio, *tokio::sync::watch*, `https://docs.rs/tokio/latest/tokio/sync/watch/index.html`.

Dokumentację zewnętrzną sprawdzono 24.10.2026. Zachowanie własnego silnika określa niniejsza specyfikacja; odnośniki *latest* służą jako kontekst, a zgodność integracji sprawdza się z przypiętymi wersjami używanymi w demonstracji.